

Image Denoising: From Foundations to State-of-the-Art

Eighteen years in which the image prior moved out of human hands — from a hand-written penalty term, into a file of learned weights, and then into a per-image optimisation that needs no clean data at all.

PRESENTERS

Navid Zare · Ali Ahmadi
Komeil Khodayar

ANCHOR PAPER

DnCNN — Zhang, Zuo, Chen,
Meng & Zhang
IEEE TIP 26(7), 2017

SOURCES

15 total · 12 Category A (80.0%)
3 Category B (20.0%) · 2007 →
2025

STRUCTURE

Four eras · three bottlenecks
One fork, in 2018

SUPERVISOR

Prof. Zohreh Azimifar
Final defence: 31 August 2026

Every claim carries its source

The deck is built to be interrogated. Each slide names where its numbers and its statements come from, so any single claim can be traced back to a primary source in one step.

-
- 01 **Paper tags** — a bracketed short name and, where relevant, the exact table: [DnCNN17 · Table II], [Restormer22 · Table 6]. Numbers reproduced from a paper are never rounded, reshaped, or averaged across papers.
 - 02 **[measured]** — figures produced by us, in a container, from the repositories themselves. Method and hardware are stated on the slide that reports them.
 - 03 **[brief]** — the assignment brief, *From Foundations to State-of-the-Art: A Comprehensive Synthesis*, Prof. Zohreh Azimifar, July 2026.
 - 04 **[reading]** — our interpretation, analogy, or teaching device. Labelled separately because it is *not* a source claim and should not be defended as one.
 - 05 Where a figure could not be verified, it is absent. No number in this deck was estimated, inferred from a plot, or carried across from a secondary citation.

One thesis, stated before the evidence

Image denoising is the cleanest available case study in how computer vision replaced hand-designed priors with learned ones — because in denoising, unusually, the prior is written down explicitly and you can watch it move.

ERA I → II

Bottleneck 1 · Manual prior design

Classical methods solve an optimisation *per image* with a hand-written penalty Φ . WNNM needs 773 s for one 512×512 image. The prior is a human guess, and the guess costs minutes.

ERA III → IV/V

Bottleneck 3 · The receptive field

A 17-layer CNN sees 35×35 pixels. It cannot exploit a repeating pattern 200 pixels away — which is exactly what non-local classical methods were good at, and exactly where DnCNN loses.

ERA II → III

Bottleneck 2 · The clean-data requirement

Discriminative learning removed test-time optimisation, but demanded paired clean/noisy corpora. For microscopy, astronomy, and medical imaging, that pairing does not exist and cannot be manufactured.

TODAY

Three live answers, not one winner

Transformers (Restormer), pure convolution done right (NAFNet), and state-space models (MambaIR) each claim the third bottleneck. The field has not settled. That is the honest ending.

reading — each bottleneck named by the paper that broke it; see Parts II–V

Eight parts, mapped onto the brief's required structure

The brief specifies seven sections. This deck follows them in order, splitting the modern era into two parts because the field genuinely forked in 2018 and collapsing that fork would misrepresent it.

PART	CONTENTS	BRIEF SECTION	SLIDES
I — The problem	Degradation model, ill-posedness, MAP, PSNR/SSIM, benchmarks	§1 Introduction & Problem Definition	06–16
II — Classical DIP	Spatial filters → wavelets → TV → NLM → BM3D → WNNM	§2 The Classical DIP Foundation	17–29
III — The transition	MLP, unrolling, CSF, TNRD, and DnCNN in full	§3 The Deep Learning Transition	30–44
IV — The data branch	Noise2Noise → Noise2Void → ZS-N2N → P2N	§4 Modern SOTA (supervision)	45–52
V — The architecture branch	Attention, Restormer, NAFNet, SCUNet, MambaIR	§4 Modern SOTA (architecture)	53–65
VI — Engineering reality	Repositories, code, pipelines, weights, measurements	§5 Practical Deep-Dive	66–80
VII — Comparative analysis	PSNR/SSIM across 18 years; efficiency; real noise	§6 Comparative Analysis	81–94
VIII — Open problems	What is still unsolved, and what we would build today	§7 Future Directions & Conclusion	95–99

Back matter — bibliography, verification notes, and the defence map: slides 98–101.

brief §3, Guidelines for Structuring the Final Slide Deck



I

The Problem

— PART I

— In this part

- 01 The degradation model
- 02 Why the problem is ill-posed
- 03 MAP and the birth of the prior
- 04 PSNR, SSIM, and what they hide
- 05 The benchmark datasets
- 06 Why denoising is foundational

What denoising is, why it is mathematically impossible without an assumption, and how we agree to measure success.

One equation, and it has never changed

Every method in this deck — from a 3×3 box filter to a state-space model — is an attempt to invert the same two-term equation. The equation is stated identically in papers sixteen years apart.

$$y = x + v$$

y — the noisy observation the sensor produced · x — the latent clean image we want · v — the noise

- 01 DnCNN, 2017: the degradation model is written $y = x + v$, with v additive white Gaussian noise of standard deviation σ . [1]
- 02 SCUNet, 2023: the same equation is written $y = x + n$, sixteen years after BM3D, in a Transformer paper. [2]
- 03 Noise2Void, 2019, states it as a *joint distribution* instead: $p(s, n) = p(s) \cdot p(n|s)$, with the signal s and noise n drawn together. That reformulation is what makes self-supervision possible — it is the same model, written so the assumptions are visible. [3]
- 04 The constancy is the point. Eighteen years of progress changed nothing about the *problem statement* — only about who supplies the missing information needed to invert it.

Additive white Gaussian noise, one word at a time

AWGN is the standard assumption, and almost every quantitative claim in this deck is made under it. It is also, as Part VII shows with hard numbers, the single assumption that breaks hardest on real photographs.

ADDITIVE

v is added, not blended

Pixel 200 becomes $200 + \epsilon$. Not multiplied (speckle), not applied through a nonlinearity (compression), not signal-dependent (shot noise). The clean image is recoverable by subtraction alone — which is precisely what makes residual learning possible in Part III.^[1]

GAUSSIAN

Zero-mean bell curve

Small errors common, large errors rare, mean zero. Zero-mean is load-bearing: it is the whole of the Noise2Noise^[3] argument. Gaussianity is what makes batch normalisation and residual learning benefit each other in DnCNN.^[1]

WHITE

Spatially independent

Each pixel's noise is independent of its neighbours'. No spatial correlation, no structure. This assumption is what Noise2Void^[4] depends on for its blind spot to work, and it is the first thing resizing destroys.

Σ

The noise level

The width of the bell. Benchmarks report $\sigma \in \{15, 25, 50\}$ on 8-bit images. DnCNN-B^[1] trains blind across $\sigma \in [0, 55]$; SCUNet^[2] samples σ from $\{2/255 \dots 50/255\}$. $\sigma = 15$ is mild grain, $\sigma = 50$ is severe.

^[1] DnCNN17 §I, §III-D ^[2] SCUNet23 §3.2 ^[3] N2N18 §2 ^[4] N2V19 §3.1

The problem is ill-posed — and no amount of compute fixes that

Given y , infinitely many pairs (x, v) satisfy $y = x + v$. Arithmetic alone cannot choose between them. Every denoiser ever written is a rule for choosing, and the quality of a denoiser is the quality of its choosing rule.

The impossibility, stated plainly

- If I tell you two numbers sum to 10, you cannot recover them. $5+5$, $3+7$, $9.7+0.3$ are all valid.
- A denoiser faces exactly that, once per pixel, with the extra constraint that the answers must be jointly plausible as a photograph.
- So the algorithm must *add information that is not in the measurement*. That added information is called the **prior**.
- There is no such thing as a prior-free denoiser. A method that claims not to have one has merely hidden it — in an architecture, in a training set, or in a stopping criterion.

Why the field is a history of priors

- Classical era: the prior is a formula a human wrote. You can read it.
- Deep era: the prior is a tensor of weights induced from data. You cannot read it, but you can download it.
- Self-supervised era: the prior is partly *the test image itself*, exploited through its own internal statistics.
- This deck's organising question, and the question Stage 2 asked of every repository: **open the source file — where is the prior?**

reading, following DnCNN17 §I on prior-based vs discriminative methods

From Bayes to the objective everyone optimises

The classical era's answer to ill-posedness is Maximum a Posteriori estimation. The derivation is three lines, and it produces the exact two-term objective that BM3D, WNNM, TV and TNRD all instantiate differently.

$$\hat{x} = \arg \max_x p(x \mid y) = \arg \max_x p(y \mid x) p(x)$$

Bayes' rule. The posterior is the likelihood times the prior.^[1]

$$\hat{x} = \arg \min_x \left[-\log p(y \mid x) - \log p(x) \right]$$

Take negative logarithms: maximising a product becomes minimising a sum.

$$\hat{x} = \arg \min_x \frac{1}{2} \|y - x\|_2^2 + \lambda \Phi(x)$$

data fidelity — stay near what the sensor saw · the prior — look like a real image · λ — how far you trust the prior

- ⁰¹ The first term is **forced**: if $v \sim N(0, \sigma^2 I)$ then $-\log p(y|x)$ is the squared error over $2\sigma^2$, with $1/\sigma^2$ absorbed into λ . The ℓ_2 data term is not a modelling choice.
- ⁰² The second term, Φ , is **entirely chosen** — and the whole classical era is a sequence of guesses at it: TV, sparse coding, weighted nuclear norm.^[2]

^[1] Standard MAP derivation ^[2] consistent with DnCNN17 SI framing of prior-based methods

From Bayes to the objective everyone optimises

- 03 λ trades them off. In the classical era it is a constant a human tunes; in the deep era it disappears, because the two terms stop being separable.
- 04 **Read the whole deck as: what is Φ , and who wrote it?**

[1] Standard MAP derivation [2] consistent with DnCNN17 §I framing of prior-based methods

PSNR — what it measures, and what it cannot

PSNR is the field's reporting currency. Every table in this deck is in decibels. Understanding its compression is the difference between reading those tables and being impressed by them.

$$\text{MSE} = \frac{1}{MN} \sum_i \sum_j [x(i, j) - \hat{x}(i, j)]^2$$

mean squared error between the estimate and the ground truth, over all MN pixels^[2]

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \text{ dB} = 20 \log_{10}(\text{MAX}) - 10 \log_{10}(\text{MSE})$$

$\text{MAX} = 255$ for 8-bit images. Higher is better. Identical images give $\text{MSE} = 0$ and $\text{PSNR} = \infty$.^[2]

- 01 **The logarithm compresses everything.** Halving the MSE gains exactly 3.01 dB, so DnCNN's 0.6 dB is about a 13% MSE reduction.
- 02 **The calibration, from the paper itself.** Prior work put the practical ceiling over BM3D at 0.3 dB and the estimated bound at 0.7 dB. DnCNN-S took 0.6 dB at all three noise levels — about 85% of the available headroom, and DnCNN-B did it blind. ^[1]
- 03 **PSNR can be gamed by blurring,** which lowers MSE while destroying texture. That is why SSIM is reported alongside it.
- 04 **It is pixel-wise and has no model of perception.** Identical PSNR can look entirely different depending on where the error sits.

SSIM — structure, not pixels

SSIM was designed to punish exactly the failure mode PSNR rewards: the loss of local structure. It decomposes similarity into three comparisons and multiplies them.

$$\text{SSIM}(x, \hat{x}) = [l(x, \hat{x})]^\alpha [c(x, \hat{x})]^\beta [s(x, \hat{x})]^\gamma, \quad \alpha = \beta = \gamma = 1$$

luminance × contrast × structure, computed in a sliding local window and averaged^[1]

$$\text{SSIM} = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

μ — local means · σ² — local variances · σ_xσ_ŷ — local covariance · C₁, C₂ — small stabilising constants^[1]

- 01 Range is [0, 1]; 1 means identical. Reported to three decimals — Restormer's real-noise SSIM on SIDD is 0.960. ^[3]
- 02 **The structure term is a local correlation coefficient.** Blur decorrelates the estimate from the reference, so blurring lowers SSIM even where it raises PSNR. That is the entire reason to report both.^[2]
- 03 SSIM correlates better with human judgement than PSNR, but it is still a hand-designed perceptual model. Modern work adds LPIPS and no-reference metrics.
- 04 **On IoU.** The brief lists it because the brief is written generically across many topics. IoU measures region overlap and is undefined for dense regression with continuous output. We report PSNR and SSIM, and say why. [\[brief §3.6\]](#)

^[1] Wang et al., IEEE TIP 2004 (standard SSIM) ^[2] brief §3.6 ^[3] Restormer22 Table 6

Nine datasets, and what each one is actually testing

Benchmark choice is not neutral. Each dataset in this deck was chosen by its authors to expose a specific weakness, and the differences between them carry most of the argument in Part VII.



Figure 1. The 20 test images.

The 20 standard grayscale/colour test images used across the classical era — Cameraman, Barbara, Lena and the rest. [WNNM14 · Fig. 1]

DATASET	CONTENT	WHAT IT STRESSES	USED BY[1][2][3][4]
BSD68	68 natural grayscale images, 481×321	General-purpose average case. The standard denoising benchmark since 2007.	DnCNN, TNRD, N2V, Restormer
Set12	12 classics — Cameraman, House, Barbara, Lena...	Per-image behaviour. Barbara and House are dominated by repetitive texture.	DnCNN Table III, Restormer Table 4
Urban100	100 urban photographs, high resolution	Long-range self-similarity. Building façades repeat identically across hundreds of pixels.	Restormer, MambaIR, SwinIR

[1] DnCNN17 §IV [2] Restormer22 §4.4 [3] MambaIR24 §4.1 [4] NAFNet22 §5

Nine datasets, and what each one is actually testing

DATASET	CONTENT	WHAT IT STRESSES	USED BY ^{[1][2][3][4]}
CBSD68 / Kodak24 / McMaster	Colour natural images	Cross-channel noise correlation and colour artefacts.	Restormer Table 5, MambaIR Table 5
SIDD	Smartphone images, real sensor noise, 320 training scenes	The AWGN assumption itself. Noise is signal-dependent and processed by an ISP.	NAFNet, Restormer, MambaIR, P2N
DND	Real photographs, held-out benchmark server	Generalisation — no training data is released, so you cannot overfit it.	Restormer, MambaIR
Set5 / Set14 / BSD100	Super-resolution benchmarks	Whether a denoiser generalises to other inverse problems.	DnCNN-3, MambaIR
Classic5 / LIVE1	JPEG-compressed images	Structured, non-Gaussian residual.	DnCNN-3 Table V

Urban100 and SIDD are the two datasets that carry the argument. Urban100 exposes bottleneck 3; SIDD exposes the AWGN assumption. Both appear again in Part VII with numbers.

Denoising is rarely the product — it is the substrate

The strongest argument for studying denoising is not that grainy photos are annoying. It is that denoising is the canonical inverse problem, and a good denoiser turns out to be a reusable component in problems that are not denoising at all.

REASON 1

It evaluates priors

SCUNet's opening argument: denoising *can help to evaluate the effectiveness of different image priors and optimization algorithms*. It is the cleanest possible test bed because the degradation is exactly known. [1]

REASON 3

It is the first step for vision

Denoising *could be the very first step for other vision tasks*. Detection, segmentation and classification all degrade on noisy input; the denoiser is the preprocessing that makes the downstream stack work at all. [1]

REASON 2

It plugs into other problems

A denoiser can be inserted into variable-splitting algorithms — half-quadratic splitting, ADMM — to solve deblurring and super-resolution. This is the *plug-and-play prior* result: solve denoising once, get a general-purpose regulariser. [1]

REASON 4

One model, three tasks

DnCNN proved that super-resolution and JPEG deblocking are *special cases of a general image denoising problem* — redefine v and the same network and loss solve all three. Denoising is not one task; it is a template. [2]

[1] SCUNet23 §1 [2] DnCNN17 §III-D

II

The Classical DIP Foundation

PART II · 2005 – 2014

In this part

- 01 Spatial filters and the edge problem
- 02 Transform-domain shrinkage
- 03 Total variation
- 04 Non-local self-similarity
- 05 BM3D, in full
- 06 WNNM and the classical ceiling
- 07 Bottleneck 1

Nine years in which the prior was a formula a human wrote down, and the answer to every image was recomputed from scratch.

Averaging: why it works, and exactly how much

The founding intuition of the entire field. If noise is zero-mean and independent, averaging n observations of the same quantity divides the noise standard deviation by $\sqrt{n}$. Every classical method is a different answer to the question: *which pixels count as observations of the same quantity?*

$$\hat{x}(i, j) = \frac{1}{|W|} \sum_{(p, q) \in W} y(p, q)$$

the box / arithmetic mean filter over a neighbourhood W centred on (i, j) ^[1]

$$\text{Var} [\hat{x}] = \frac{\sigma^2}{|W|} \implies \sigma_{\text{out}} = \frac{\sigma}{\sqrt{|W|}}$$

a 3×3 window divides noise by 3; a 5×5 window divides it by 5^[1]

- ⁰¹ The derivation only holds if every pixel in W has the *same underlying clean value*. That assumption is true inside a flat region and catastrophically false across an edge.
- ⁰² Across an edge, the mean of the window is not the value of the centre pixel, so the filter introduces a **bias** exactly proportional to the local contrast. You trade variance (noise) for bias (blur), pixel by pixel, and a box filter makes that trade blindly.

Averaging: why it works, and exactly how much

- 03 This is the bias–variance trade-off in its purest visual form: the quantity you reduce is visible as grain, and the quantity you increase is visible as softness.
- 04 **The entire classical era is a search for better definitions of W .**^[2] Gaussian weights it by distance. Bilateral weights it by distance *and* intensity. Non-local means abandons locality entirely and weights by patch similarity. BM3D stops weighting and starts *stacking*.

Gaussian, median, and the first hard-coded threshold

Two filters, two different failure modes, and the first appearance of the problem the brief names explicitly: reliance on hard-coded parameters.

Gaussian filter — linear, isotropic

- Weights $w(p,q) \propto \exp(-[(p-i)^2 + (q-j)^2] / 2\sigma_s^2)$. Nearer pixels count more.
- Separable: an $N \times N$ Gaussian is two $1 \times N$ passes, so cost is $O(N)$ per pixel, not $O(N^2)$. This is why it was the default for decades.
- It is the Green's function of the heat equation — Gaussian smoothing *is* isotropic diffusion run for time $t = \sigma_s^2/2$. That equivalence is what TNRD later generalises, in Part III.^[1]
- **Failure mode:** it diffuses across edges as happily as along them. Structure and noise are treated identically because the weights depend only on geometry.^[2]

Median filter — nonlinear, order-statistic

- Replace the centre with the *median* of the window. Not a convolution — no kernel exists.
- Optimal for impulse (salt-and-pepper) noise, because an outlier cannot move a median the way it moves a mean. This is the same reason Noise2Noise's ℓ_1 loss recovers a median, in Part IV.^[1]
- Preserves step edges exactly: the median of a half-black, half-white window is black or white, never grey.
- **Failure mode:** destroys thin lines and corners, and is poorly matched to Gaussian noise, where the mean is the optimal estimator, not the median.^[2]

Both filters require a window size chosen by a human, in advance, per image. The brief names this directly as a limitation of the classical era: *reliance on hard-coded thresholds*.

^[1] G&W, *Image Restoration* chapter ^[2] brief §3.2

Sparsity: the idea that a clean image is compressible and noise is not

The second great classical strategy. Move to a basis where the signal concentrates into a few large coefficients while the noise stays spread thinly across all of them — then discard the small ones.

$$\text{hard: } T_h(c) = c \cdot \mathbf{1}[|c| > \lambda] \qquad \text{soft: } T_s(c) = \text{sign}(c) \max(|c| - \lambda, 0)$$

c — a transform coefficient · λ — the shrinkage threshold, typically $\lambda = \sigma\sqrt{2 \log N}$

- 01 **Why it works.** An orthonormal transform preserves the noise variance — white noise stays white, spread evenly over all coefficients at amplitude $\approx \sigma$. A natural image, in a wavelet or DCT basis, concentrates its energy into few large coefficients. Thresholding therefore removes far more noise than signal.
- 02 **Hard vs soft.** Hard thresholding is unbiased on surviving coefficients but discontinuous, producing ringing artefacts. Soft thresholding is continuous and produces smoother output but biases every surviving coefficient toward zero by λ . BM3D uses hard thresholding in step 1 and a Wiener filter in step 2 precisely to get the benefit of each. [1]
- 03 **Where it fails.** A single fixed basis — DCT, or one wavelet family — cannot be sparse for every image structure simultaneously. Edges at arbitrary orientations are notoriously non-sparse in separable wavelets, which is exactly where ringing appears.
- 04 **The escape.** If a fixed basis is not sparse enough, learn one. That is K-SVD (Elad & Aharon, IEEE TIP 2006)[2]: learn an overcomplete dictionary D from the image's own patches and represent each patch as $x \approx D\alpha$ with $\|\alpha\|_0$ small. The prior is still hand-designed — sparsity — but the *basis* is now data-driven. This is the first crack in the classical wall.

[1] BM3D07 §III [2] Elad & Aharon, IEEE TIP 2006 (cited in DnCNN17)

ROF: the first prior that treats edges as information rather than error

Rudin, Osher and Fatemi, 1992. Instead of penalising roughness with a squared gradient — which is quadratic and therefore hates edges — penalise the ℓ_1 norm of the gradient, which is cheap for a few large jumps and expensive for pervasive ripple.

$$\hat{x} = \arg \min_x \frac{1}{2} \|y - x\|_2^2 + \lambda \|\nabla x\|_1, \quad \|\nabla x\|_1 = \sum_{i,j} \sqrt{(\partial_h x)^2 + (\partial_v x)^2}$$

the isotropic total-variation semi-norm — the ℓ_1 norm of the gradient magnitude field

- 01 **Why ℓ_1 and not ℓ_2 .** A Tikhonov prior $\|\nabla x\|_2^2$ charges the *square* of a jump, so one edge of height 100 costs 10 000 while a hundred ripples of height 1 cost only 100 — the optimiser removes the edge.^[1] TV charges the jump itself, so an edge of height 100 costs 100 and a hundred ripples also cost 100. Edges become affordable.
- 02 **Consequence — and the classic artefact.** TV's minimiser is piecewise constant. It preserves edges beautifully and turns smooth gradients into visible flat plateaus. This is *staircasing*, and it is the reason TV is not a general-purpose denoiser.
- 03 **Non-differentiability.** $\|\cdot\|_1$ has no gradient at zero, so the objective cannot be minimised by plain gradient descent. Chambolle's^[2] 2004 dual projection algorithm solves it by iterating on the dual variable to a fixed tolerance.
- 04 **The engineering consequence, which matters in Part VI.** The algorithm is a `while` loop with a tolerance and a maximum iteration count, run *per image*. The prior is one line of arithmetic you can read; the price is that nothing is ever reused.

^[1] Rudin, Osher & Fatemi 1992 ^[2] Chambolle 2004 ^[3] implementation inspected: `scikit-image 0.26.0 `denoise_tv_chambolle` [measured]`

Non-local self-similarity — stop looking nearby

Buades, Coll and Morel, CVPR 2005. Every method so far assumed the pixels that inform a pixel are the pixels *near* it. Natural images violate that assumption constantly: a brick resembles the other bricks, a stripe resembles the other stripes, and those twins may be two hundred pixels away.

Non-local means

- $\hat{x}(i) = \sum_j w(i,j) \cdot y(j)$, summed over the whole image, with $w(i,j) \propto \exp(-\|P_i - P_j\|^2_a / h^2)$. [1]
- The weight compares *patches* P centred on i and j , not single pixels. Two pixels are similar if their neighbourhoods are similar.
- Averaging many independent observations of the same structure cancels the noise while the shared signal survives — the $\sqrt{n}$ argument from slide 15, now applied to patches rather than a square window.
- Cost is quadratic in the number of pixels unless the search is restricted to a window, which every practical implementation does.

Why this matters for the whole deck

- NSS is the strongest hand-designed prior ever found for natural images. It powers BM3D and WNNM, and it is the *specific* prior DnCNN lacks.
- WNNM's own framing: NSS refers to the fact that there are many repeated local patterns across a natural image, and those non-local similar patches to a given patch can help much the reconstruction of it. [2]
- This single idea explains the two most quoted results in this deck — DnCNN's loss on Barbara (Part III) and Restormer's enormous gain on Urban100 (Part VII).
- It is also the reason the third bottleneck exists. Convolution is local by construction; recovering non-locality inside a neural network took from 2017 to 2022.

[1] Buades et al., CVPR 2005 (cited in DnCNN17) [2] WNNM14 §3

The algorithm that defined the benchmark for a decade

Dabov, Foi, Katkovnik and Egiazarian, IEEE TIP 2007. BM3D combines non-local grouping with transform-domain shrinkage, and runs the combination twice. Its two-step structure is stated verbatim in the paper.

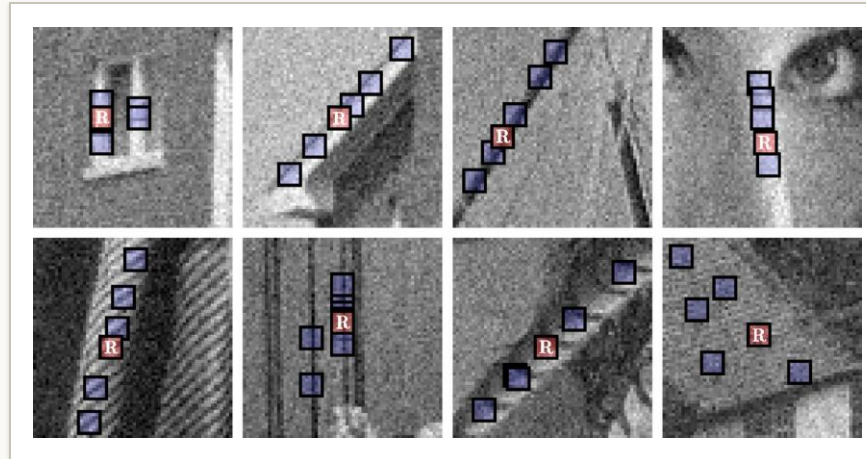
■ Step 1a-i · Grouping	For each reference block in the noisy image, find blocks similar to it by block-matching and stack them together in a 3-D array — a <i>group</i> .
■ Step 1a-ii · Collaborative hard-thresholding	Apply a 3-D transform to the group, attenuate the noise by hard-thresholding the coefficients, invert the transform, and return the estimates to their original positions.
■ Step 1b · Aggregation	Compute the basic estimate by weighted averaging of all overlapping block-wise estimates.
■ Step 2a-i · Re-grouping	Use block-matching <i>within the basic estimate</i> — cleaner, so matching is more accurate. Form two groups: one from the noisy image, one from the basic estimate.
■ Step 2a-ii · Collaborative Wiener filtering	Wiener-filter the noisy group using the energy spectrum of the basic estimate as the pilot spectrum.
■ Step 2b · Aggregation	Aggregate the local estimates by weighted average into the final estimate.

⁰¹ The paper gives two motivations for the second step explicitly: using the basic estimate rather than the noisy image **improves the grouping**, and using it as the Wiener pilot signal is **much more effective and accurate than simple hard-thresholding** of the noisy spectrum. [BM3D07 §II-C]

BM3D07 §II-C, algorithm stated verbatim

Why *collaborative* filtering beats filtering each patch alone

The word 'collaborative' is doing real work. Stacking similar patches into a 3-D array creates sparsity along a third axis that does not exist in any single patch — and that third axis is where nearly all the denoising happens.



Block matching in practice: the reference patch R (red) and the similar patches BM3D groups with it before collaborative 3-D filtering. [BM3D07 · Fig. 2]

$$\hat{Y}_S = T_{3D}^{-1} \left(\gamma(T_{3D}(Y_S)) \right), \quad \gamma(c) = c \cdot \mathbf{1} \left[|c| > \lambda_{3D} \sigma \right]$$

Y_S — the 3-D group of matched blocks · T_{3D} — a separable 3-D transform · γ — hard thresholding

Why *collaborative* filtering beats filtering each patch alone

$$T_{3D} = T_{2D} \otimes T_{1D}$$

a 2-D transform within each block (intra-fragment), composed with a 1-D transform across the stack (inter-fragment)

- 01 **The paper's own ablation is the evidence.** Varying the two transforms changes PSNR only modestly — even a basis that is *random apart from the DC* loses just 0.1–0.4 dB. **Inter-fragment correlation matters far more than intra-fragment correlation.** [BM3D07 §V]
- 02 **Which is why the DC element matters.** The DST performs worst of every transform tested, and the paper blames its *lack of a DC basis element* — the DC along the stacking axis is what captures similarity, recovering plain averaging as a special case.
- 03 **Complexity** is linear in image size; cost per pixel is block matching plus separable transforms. The Fast profile swaps exhaustive for predictive search.
- 04 **Parameters:** block size, search window, matching threshold, λ_3D , two transforms, aggregation weights — all author-set, with different values above and below $\sigma = 40$.

Why BM3D was the benchmark, not merely a benchmark

For roughly a decade every denoising paper had to beat BM3D, and almost none did by a meaningful margin. Three numbers explain why that stalemate lasted.

0.3 dB

THE PRACTICAL CEILING

Prior work found that *few methods can outperform BM3D by more than 0.3 dB on average*, a figure DnCNN quotes when calibrating its own result. [1]

0.7 dB

THE ESTIMATED THEORETICAL BOUND

The estimated PSNR bound over BM3D reported in the literature — Levin & Nadler's work on the inherent limits of natural-image denoising. [1]

2.85 s

512×512, CPU, $\Sigma = 25$

Fast enough to be usable, unlike WNNM's 773 s. BM3D occupied the only viable point on the quality-vs-time curve. [1]

- 01 BM3D's scores on BSD68 — 31.07 / 28.57 / 25.62 dB at $\sigma = 15 / 25 / 50$ — are the reference line for every table in Part VII. [1]
- 02 **And it is still in use.** BM3D appears as a live baseline in Noise2Noise (2018), Noise2Void (2019), ZS-N2N (2023) and Restormer (2022). [2] A seventeen-year-old algorithm is still the thing new zero-shot methods must beat, because it requires no training data at all.
- 03 **The honest framing for the defence:** BM3D was not superseded because it was bad. It was superseded because it was *the best a human could do by hand*, and the next gain had to come from somewhere other than a human.

[1] DnCNN17 §IV-B, Table II, Table IV [2] baselines in N2N18, N2V19, ZS-N2N23, Restormer22

The classical peak: low-rank recovery on stacks of similar patches

Gu, Zhang, Zuo and Feng, CVPR 2014. WNNM takes the same non-local grouping as BM3D but replaces transform shrinkage with low-rank matrix approximation — and then fixes the standard low-rank estimator so it respects the physics of denoising.

$$Y_j = X_j + N_j, \quad X_j \text{ low rank}$$

Y_j — a matrix whose columns are the nonlocal similar patches to patch j · X_j — their clean counterpart

$$\hat{X}_j = \arg \min_{X_j} \frac{1}{\sigma_n^2} \|Y_j - X_j\|_F^2 + \|X_j\|_{w,*}, \quad \|X\|_{w,*} = \sum_i w_i \sigma_i(X)$$

first term: Frobenius-norm data fidelity, normalised by the noise variance · second term: the weighted nuclear norm — a differently weighted sum of singular values, not an equally weighted one

- 01 **The setup.** Stacking similar patches makes the matrix low rank, because the columns are near-copies. Low-rank recovery from a noisy matrix is a well-studied convex problem.
- 02 **The defect WNNM fixes.** Standard nuclear-norm minimisation shrinks every singular value by the same λ . The paper argues **the larger ones carry the major image components and should be shrunk less.**
- 03 **The weighting rule:** $w_i = c \cdot \sqrt{n} / (\sigma_i(X_j) + \epsilon)$, inversely proportional to the singular value — large components barely touched, small ones crushed.
- 04 **The circularity.** The weights need the unknown clean singular values; WNNM estimates them assuming noise energy spreads evenly across subspaces.

The theorem that makes it tractable, and the price it charges

Weighting the nuclear norm destroys convexity in general. WNNM's contribution is a theorem identifying exactly when a closed-form solution survives, plus an iterative algorithm for when it does not.

- 01 **Theorem 2 (WNNM).** If the weights satisfy $w_1 \geq w_2 \geq \dots \geq w_n \geq 0$ — that is, *non-ascending* order — the problem has a globally optimal solution obtained by generalised soft-thresholding of the singular values of Y with the weight vector w : $\hat{X} = U S_w(\Sigma) V^T$. The classical singular value thresholding result of Cai et al. is recovered as the special case $w_1 = \dots = w_n = \lambda$.
- 02 **The catch for denoising.** The denoising weights are *non-descending* (small singular values get large weights), which is the opposite ordering. The paper therefore proves a separate result: when weights are in non-descending order, an iterative algorithm reaches an analytical **fixed point** in one step, because soft-thresholding preserves the non-ascending order of the singular values, so the auxiliary permutation matrices stay at identity.
- 03 **Result.** WNNM leads to *visible PSNR improvements over state-of-the-art methods such as BM3D*, and preserves local structure with fewer visual artefacts. On BSD68 it scores 31.37 / 28.83 / 25.87 dB at $\sigma = 15 / 25 / 50$, beating BM3D at every level. [1][2]
- 04 **The price, and it is enormous.** WNNM requires an SVD of every patch group, iterated, per image. Measured in the DnCNN paper's own comparison: **773.2 seconds for a single 512×512 image**, and 2 536 seconds at 1024×1024 — against BM3D's 2.85 s and 12.89 s. [2]
- 05 **Effective patch size 361 × 361.** WNNM consults a region more than ten times wider than DnCNN's 35×35 receptive field. Hold on to that number — it reappears on slide 55 and it is the reason WNNM still wins on Barbara.

[1] WNNM14 Theorem 2, §2.3, abstract [2] DnCNN17 Table I, Table II, Table IV

Where nine years of hand-designed priors finished

All figures below are reproduced from Table II and Table IV of the DnCNN paper, which benchmarked every classical competitor under one protocol on one dataset. Reading them across a single row is the fairest available comparison of the era.

METHOD	YEAR	PRIOR Φ	BSD68 $\Sigma=15$	$\Sigma=25$	$\Sigma=50$	512 ² RUNTIME
BM3D	2007	NSS + 3-D transform sparsity	31.07	28.57	25.62	2.85 s CPU
EPLL	2011	Gaussian mixture patch prior	31.21	28.68	25.67	45.5 s CPU
CSF	2014	Unrolled field of experts	31.24	28.74	—	5.67 s / 0.92 s GPU
WNNM	2014	NSS + weighted low rank	31.37	28.83	25.87	773.2 s CPU
MLP	2012	learned (plain network)	—	28.96	26.03	5.51 s CPU
TNRD	2016	Unrolled reaction–diffusion	31.42	28.92	25.97	1.33 s / 0.032 s GPU

Two readings. (a) Nine years of work moved BSD68 $\sigma=25$ from 28.57 to 28.83 — **0.26 dB**. (b) The best classical method is 271× slower than the second best. The era had hit a wall in both quality and cost simultaneously.

DnCNN17 Table II and Table IV — all values as printed

Why the field had to leave — in the words of the paper that left

DnCNN's introduction names the classical era's drawbacks precisely. Quoting the paper that broke a bottleneck, rather than asserting the bottleneck ourselves, is what makes this timeline evidenced rather than imposed.

DRAWBACK 1

Test-time optimisation

Prior-based methods *involve a complex optimization problem in the testing stage*, making the denoising process time-consuming. Every image restarts from nothing; nothing learned on image 1 helps image 2. [DnCNN17 §I]

DRAWBACK 3

The prior's form limits the result

Even for the discriminative bridge methods, performance is *inherently restricted to the specified forms of prior*; analysis-model priors are *limited in capturing the full characteristics of image structures*. [DnCNN17 §I]

DRAWBACK 2

Non-convex, hand-tuned

The models are *generally non-convex and involve several manually chosen parameters*. Reproducibility and deployment both suffer: the quality you get depends on constants a human picked. [DnCNN17 §I]

DRAWBACK 4

One model per noise level

Bridge methods *train a specific model for a certain noise level* and are *limited in blind image denoising*. In a phone camera, σ is never known. [DnCNN17 §I]

Bottleneck 1, stated once: the prior is written by hand, and paid for once per image. The next era removes both halves of that sentence.

DnCNN17 §I — all four drawbacks quoted from the paper's introduction

III

The Deep Learning Transition

— PART III · 2012 – 2017

— In this part

- 01 MLP — the first plain network to matter
- 02 Unrolling as the conceptual bridge
- 03 CSF and TNRD
- 04 DnCNN: architecture, residual learning, BN synergy
- 05 Depth, receptive field, data, results
- 06 One model, three tasks
- 07 Bottleneck 2

Five years in which the optimisation loop was unrolled into a fixed architecture, and then abandoned entirely.

MLP, 2012 — a plain neural network reaches BM3D

Burger, Schuler and Harmeling, CVPR 2012. Before any architectural insight, before residual learning, before batch normalisation: a multi-layer perceptron trained on enough patches matched the best hand-designed algorithm in the field.

- 01 **The method is deliberately unsophisticated.** Take a noisy patch, flatten it to a vector, push it through several fully connected layers^[1], and read a clean patch out. No convolution, no weight sharing, no prior of any kind written down.
- 02 **The result.** On BSD68 at $\sigma = 25$ it scores **28.96 dB**, ahead of BM3D's 28.57 and ahead of WNNM's 28.83 — the strongest result in the whole classical table at that noise level. At $\sigma = 50$ it scores 26.03, again the best pre-DnCNN figure. ^[2]
- 03 **Effective patch size 47×47 ,** arising from a 39×39 input patch followed by a 9×9 averaging filter. Comparable to BM3D's 49×49 — the network was not seeing more than its competitors. ^[2]
- 04 **What it proved, and what it did not.** It proved that a learned mapping could match a decade of hand-designed mathematics. It did not explain *why*, did not generalise across noise levels — a separate model per σ — and its fully connected structure scaled badly with patch size.
- 05 **The reading.** MLP is the existence proof, not the solution. It told the field that the answer was learning, and left the question of *what to learn and how* wide open for the next five years.

^[1] Burger et al., CVPR 2012 (cited in DnCNN17) ^[2] DnCNN17 Table I, Table II

Unrolling — turn the iterations into layers

The elegant idea that connects the two eras. A classical denoiser is an iterative optimiser: repeat some update rule K times. Unrolling fixes K , gives every iteration its own parameters, and trains the whole stack end-to-end by back-propagation.

Before — an optimiser

- Run until convergence, or until a tolerance is met. K is unbounded and data-dependent.^[2]
- Every stage applies the *same* hand-written rule with the *same* hand-tuned constants.
- Quality is whatever the fixed objective's minimiser happens to be. You cannot ask for better without redesigning Φ .
- Cost is unpredictable: a hard image takes longer than an easy one.

After — an architecture

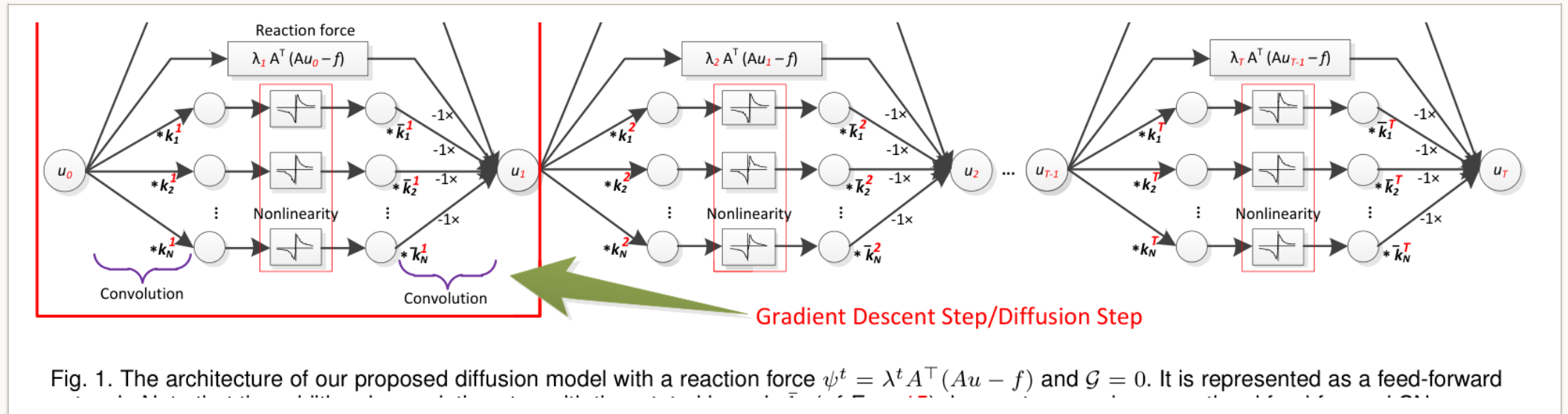
- Truncate to a fixed K — TNRD uses *usually less than 10*^[1] stages. Now the runtime is a constant.
- Give each stage its *own* filters and nonlinearities, and let them differ across stages.
- Train all of them against a loss measured on the final output. The objective being minimised is no longer the objective being optimised at test time — it is the training loss.
- The result is, structurally, a neural network — but one whose layers are inherited from a PDE rather than invented.

Why this matters for the narrative. Unrolling is the bridge that makes the classical→deep transition continuous rather than a rupture. TNRD is a diffusion equation and a CNN simultaneously, which is exactly why DnCNN can later derive itself from a one-stage TNRD.

^[1] TNRD16 §1.2, §2 ^[2] reading

Trainable Nonlinear Reaction Diffusion — a PDE with learned parameters

Chen and Pock, IEEE TPAMI 2016/2017. DnCNN's direct theoretical ancestor. The model is a classical nonlinear diffusion process in which every component — the filters *and* the influence functions — is learned from data rather than chosen.



The diffusion network unrolled as a feed-forward graph: each stage applies the learned filters, the trainable nonlinearity, and the reaction force in sequence.

[TNRD16/17 · Fig. 1]

Trainable Nonlinear Reaction Diffusion — a PDE with learned parameters

$$\frac{u_t - u_{t-1}}{\Delta t} = - \sum_{i=1}^{N_k} (K_i^t)^\top \phi_i^t(K_i^t u_{t-1}) - \psi^t(u_{t-1}, f)$$

left group = diffusion term · right group = reaction term · K_i — sparse matrices implementing 2-D convolution with kernel k_i · ϕ_i — influence functions · f — the degraded input

$$\psi^t(u) = \nabla_u D_t(u, f) = \lambda^t A^\top (Au - f), \quad A = I \text{ for denoising}$$

the reaction term is the gradient of a data-fidelity term — which is what makes one framework serve denoising, super-resolution and JPEG deblocking

- 01 **$\Delta t = 1$ in practice**, because ϕ and ψ can be freely rescaled.
- 02 **What is learned:** earlier diffusion models fixed the filters and chose the diffusivity. TNRD learns both *simultaneously* from a loss, and lets them **vary across stages**.[\[1\]](#)
- 03 **Configurations:** 24 filters of 5×5, 48 of 7×7, 80 of 9×9 — the count is m^2-1 for an $m \times m$ kernel.
- 04 **The payoff:** truncation to a few stages is *well-suited for parallel computation on GPUs*, in explicit contrast to BM3D.[\[1\]](#)
- 05 **Result and limit.** 31.42 / 28.92 / 25.97 dB[\[2\]](#) on BSD68 at 0.032 s per 512×512 on GPU — but one model per noise level, and a 61×61 effective patch size.

[\[1\]](#) TNRD16 abstract, §1, §2, §3 [\[2\]](#) DnCNN17 Tables I, II, IV

DnCNN — the four questions it set out to answer

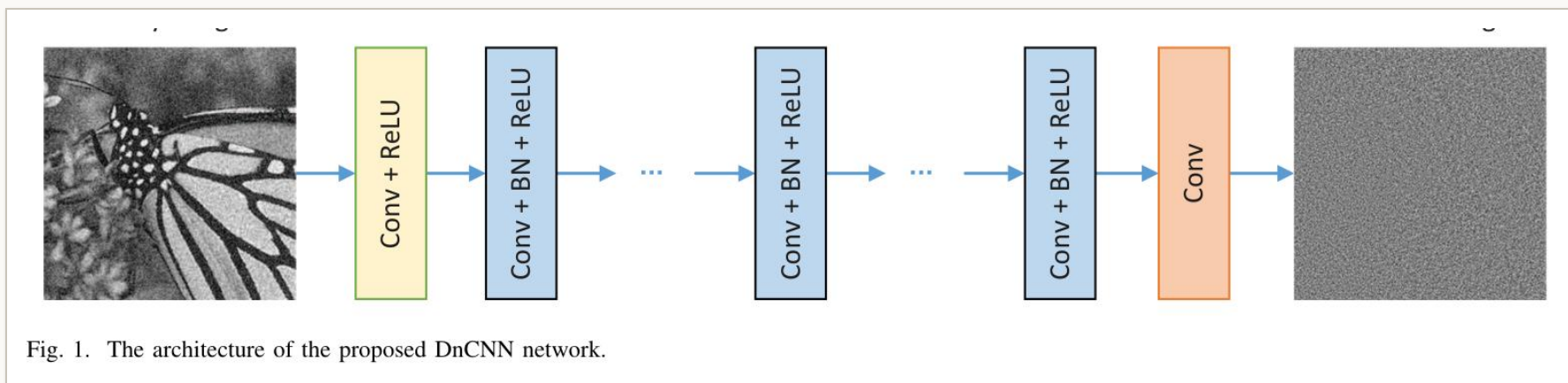
Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising. Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, Lei Zhang. IEEE Transactions on Image Processing 26(7), 2017. arXiv:1608.03981.

- 01 **Q1 — Can a deep CNN with residual learning and batch normalisation beat the state of the art on Gaussian denoising?** Answer: yes, by 0.6 dB over BM3D at all three noise levels.^[1]
- 02 **Q2 — Can a single model handle unknown noise levels?** Answer: yes. DnCNN-B trains across $\sigma \in [0, 55]$ and beats noise-level-specific competitors without being told σ .
- 03 **Q3 — Can one network handle denoising, super-resolution and JPEG deblocking together?** Answer: yes. DnCNN-3 beats a specialist baseline on all three.
- 04 **Q4 — Do residual learning and batch normalisation benefit each other?** Answer: yes, and this is the paper's most-cited empirical finding. Neither works nearly as well alone.
- 05 **The three contributions, in the authors' framing.** (1) An end-to-end trainable deep CNN that, *unlike existing deep methods which directly estimate the latent clean image*, adopts residual learning to remove the latent clean image from the noisy observation. (2) The finding that residual learning and batch normalisation *greatly benefit CNN learning*, speeding up training and boosting performance. (3) That DnCNN extends easily to general image denoising — one model for blind Gaussian denoising, one model for three tasks.
- 06 **Institutions.** Harbin Institute of Technology, The Hong Kong Polytechnic University, Xi'an Jiaotong University. **Citations: 11 674**^[2] (Google Scholar, checked 9 August 2026).

^[1] DnCNN17 abstract, §1 ^[2] citation count: Google Scholar, 9 Aug 2026

Three layer types, one global skip, and nothing else

The paper describes the design origin plainly: they *modify the VGG network to make it suitable for image denoising*, and set the depth from the effective patch sizes used by state-of-the-art denoising methods. Given depth D there are exactly three kinds of layer.



The DnCNN architecture: a noisy image enters a stack of Conv(+BN)+ReLU layers and the network predicts the residual noise image, not the clean image.
[DnCNN17 · Fig. 1]

■ Layer 1 · Conv + ReLU

64 filters of size $3 \times 3 \times c$ generate 64 feature maps; ReLU = $\max(0, \cdot)$ follows. $c = 1$ grayscale, $c = 3$ colour. **[1] No batch normalisation on the first layer.**

■ Layers 2 ... $D-1$ · Conv + BN + ReLU

64 filters of $3 \times 3 \times 64$. Batch normalisation sits *between* the convolution and the ReLU. Repeated $D-2$ times — fifteen times in DnCNN-S.

[1] DnCNN17 §III-B, Fig. 1 [2] parameter breakdown computed from the stated shapes

Three layer types, one global skip, and nothing else

■ Layer D · Conv only

c filters of size $3 \times 3 \times 64$ reconstruct the output. **No BN, no ReLU** — the output must be able to take negative values, because it is a noise map.

■ Global skip · $\hat{x} = y - \hat{v}$

The network outputs $\hat{v}$, the predicted *noise*. The clean image is recovered outside the network by subtraction — one skip across the whole network.

- 01 **Zero padding throughout**, keeping every middle feature map the same size as the input. The paper reports this simple strategy produces **no boundary artefacts**.
- 02 **No pooling anywhere**. Pooling discards spatial resolution, and denoising must emit one clean pixel per input pixel. Removing it is what makes the receptive-field arithmetic on slide 35 exact.
- 03 **Parameters, DnCNN-S, depth 17, grayscale**: $576 + 15 \times (3 \cdot 3 \cdot 64 \cdot 64) + 576 + 15 \times 2 \times 64 \approx \mathbf{556\ K}$. The network is one block repeated fifteen times.^[2]

^[1] DnCNN17 §III-B, Fig. 1 ^[2] parameter breakdown computed from the stated shapes

Residual learning — predict the dirt, not the photograph

The single design choice that defines the paper. Existing discriminative models learn $F(y) = x$, the clean image. DnCNN learns $R(y) \approx v$, the noise, and recovers $x = y - R(y)$.

$$\ell(\Theta) = \frac{1}{2N} \sum_{i=1}^N \left\| \mathcal{R}(y_i; \Theta) - (y_i - x_i) \right\|_F^2$$

Equation (1) of the paper — averaged squared Frobenius error between the desired residual images and the estimated ones, over N noisy–clean training patch pairs

- 01 **The target is $(y_i - x_i)$, which is the actual noise.** The loss never looks at the clean image directly. This is the one line to check when opening any denoising repository: does `forward()` return the image or the noise?
- 02 **The formal justification, in two steps.** (a) From ResNet: *when the original mapping is more like an identity mapping, the residual mapping will be much easier to optimize.* (b) The observation that closes it: the noisy observation y is *much more like the latent clean image x than it is like the residual image v* — especially at low noise. So $F(y)$ would be closer to an identity mapping than $R(y)$ is, and the residual formulation is more suitable.
- 03 **Why learning the identity is genuinely hard.** A stack of convolutions with ReLUs has no built-in way to express 'copy the input unchanged'; seventeen layers would have to conspire into an identity transform, which is a narrow and awkward point in parameter space. A network that outputs *zero*, by contrast, is trivially reachable — drive the weights toward zero. Residual learning relocates the 'do nothing' solution from the hard point to the easy one.
- 04 **DnCNN is not ResNet.** The paper is explicit: *unlike the residual network, which uses many residual units (identity shortcuts), DnCNN employs a single residual unit to predict the residual image.* One global skip, not dozens of local ones.
- 05 **Priority, stated with a hedge — reproduce the hedge.** Residual prediction had already been used in super-resolution and colour demosaicking, but *to the best of our knowledge there is no work which directly predicts the residual image for denoising.*

Residual learning and batch normalisation need each other

The paper's most cited empirical finding, and a frequently discussed result. The claim is a two-way street: *the integration of residual learning and batch normalization can result in fast and stable training and better denoising performance, and the two benefit from each other.*

Direction 1 — RL benefits from BN

- The straightforward half. BN alleviates internal covariate shift: as training updates early layers, the distribution arriving at later layers keeps moving, and those layers chase a target that will not hold still.
- BN normalises activations across the mini-batch before the nonlinearity, then applies a learned scale and shift — only two parameters per activation, trained by back-propagation.
- The paper's Fig. 2 shows residual learning *without* BN converges fast but lands below residual learning *with* BN.
- Benefits the paper lists: fast training, better performance, low sensitivity to initialisation.

Direction 2 — BN benefits from RL (the surprising half)

- Without residual learning, batch normalisation *even has a certain adverse effect on convergence*. BN alone is worse than nothing.
- **The mechanism.** A mini-batch is 128 patches. Without residual learning the layer inputs are correlated with their neighbours, so their distribution depends on the *content of the images in that particular batch*.
- With residual learning DnCNN *implicitly removes the latent clean image in the hidden layers*, making layer inputs **Gaussian-like distributed, less correlated, and less related to image content** — so RL helps BN reduce covariate shift.
- **The deeper reason:** *the residual image and batch normalization are both associated with the Gaussian distribution*, so it is very likely they benefit from each other.

Not the optimiser. The paper checks this: both SGD and Adam give the best results with RL+BN. It is *the integration of residual learning and batch normalization, rather than the optimization algorithm*, that produces the win.

Why 17 layers — a hyperparameter with a documented derivation

Because pooling is removed and all filters are 3×3 , the receptive field of a depth- d DnCNN is exactly $(2d+1) \times (2d+1)$. The authors chose d by anchoring that number to the *effective patch size* of competing methods at $\sigma = 25$.

METHOD	EFFECTIVE PATCH SIZE	HOW IT ARISES
EPLL	36×36	Smallest of the group
DnCNN-S	35×35	Depth 17, chosen to sit near the smallest
MLP	47×47	39×39 patch followed by a 9×9 averaging filter
BM3D	49×49	Non-local search in a 25×25 window, performed twice
CSF	61×61	Ten conv layers with 7×7 filters over five stages
TNRD	61×61	Same construction as CSF
WNNM	361×361	Large window, iterated non-local search

The design question, and it is elegant: *it is interesting to verify whether DnCNN with a receptive field size similar to EPLL can compete against the leading denoising methods. For a known σ they set RF = 35×35 , depth 17; for other general denoising tasks they adopt a larger field with depth 20 (41×41) — because *high noise level usually requires larger effective patch size*.*

The pipeline that made it work

This is the slide that answers the brief's Stage 2 question — *how do their data pipelines work* — for the anchor paper itself, and the baseline against which the modern pipelines in Part VI are contrasted.

MODEL ^[1]	SOURCE IMAGES	PATCHES	PATCH SIZE	NOISE / TASK	DEPTH
DnCNN-S	400 images at 180×180	128 × 1 600	40 × 40	σ fixed at 15, 25 or 50	17
DnCNN-B	400 images at 180×180	128 × 3 000	50 × 50	σ sampled from [0, 55]	20
CDnCNN-B	432 BSD colour images	128 × 3 000	50 × 50	σ sampled from [0, 55]	20
DnCNN-3	91 images + 200 BSD	128 × 8 000	50 × 50	Denoising + SISR + JPEG	20

Hyperparameters. MSE on the residual · SGD with momentum 0.9 and weight decay $1e^{-4}$ · learning rate decayed exponentially from $1e^{-1}$ to $1e^{-4}$ over 50 epochs · mini-batch 128 · He initialisation · **MatConvNet**, a MATLAB library. Trained on an Intel i7-5820K @ 3.30 GHz with an Nvidia Titan X: **≈ 6 hours** for DnCNN-S, **one day** for DnCNN-B/CDnCNN-B, **three days** for DnCNN-3.

^[1] DnCNN17 §IV-A ^[2] training-time figures from §IV-A

BSD68 — the headline benchmark

Average PSNR in dB, reproduced from Table II of the paper. The classical block is the same one shown on slide 25; DnCNN's two rows are now added beneath it.

METHOD	$\Sigma = 15$	$\Sigma = 25$	$\Sigma = 50$
BM3D (2007)	31.07	28.57	25.62
EPLL (2011)	31.21	28.68	25.67
MLP (2012)	—	28.96	26.03
CSF (2014)	31.24	28.74	—
WNNM (2014)	31.37	28.83	25.87
TNRD (2016)	31.42	28.92	25.97
DnCNN-S (known σ)	31.73	29.23	26.23
DnCNN-B (blind)	31.61	29.16	26.23

How to read 0.6 dB without sounding naive. Prior work found *few methods can outperform BM3D by more than 0.3 dB on average*; MLP and TNRD achieved about 0.35 dB. DnCNN-S outperforms BM3D by **0.6 dB at all three noise levels**, and at $\sigma = 50$ both variants exceed BM3D by about 0.6 dB — very close to the estimated PSNR bound over BM3D of 0.7 dB. **DnCNN captured roughly 85% of the theoretically available headroom — and DnCNN-B did it without being told σ .**

Where it loses — and why losing is the interesting result

Per-image PSNR at $\sigma = 25$, from Table III. DnCNN-S wins on ten of twelve images. The two it loses are not random, and the paper's explanation of them is the most intellectually valuable passage in its experimental section.

IMAGE	BM3D	WNNM	TNRD	DNCNN-S	Δ VS BM3D
Cameraman	29.45	29.64	29.72	30.18	+0.73
House	32.85	33.22	32.53	33.06	+0.21
Peppers	30.16	30.42	30.57	30.87	+0.71
Monarch	29.25	29.84	29.85	30.28	+1.03
Parrot	28.93	29.15	29.18	29.43	+0.50
Barbara	30.71	31.24	29.41	30.00	-0.71
Boat	29.90	30.03	29.91	30.21	+0.31
Average (12)	29.97	30.26	30.06	30.44	+0.47

The paper's own explanation. DnCNN-S fails to achieve the best result on exactly the two images *dominated by repetitive structures*, consistent with a known finding: *non-local means based methods are usually better on images with regular and repetitive structures, whereas discriminative training based methods generally produce better results on images with irregular textures*. Images with regular repetitive structures *meet well with the non-local similarity prior*; irregular textures *weaken the advantage of that specific prior*.

DnCNN17 Table III, §IV-C — values as printed

Runtime — the commercial argument, stated honestly

Seconds per image at noise level 25, from Table IV. This table is the reason denoising moved into products, and it is also the table on which a careless presenter gets caught.

METHOD	256×256	512×512	1024×1024	DEVICE
BM3D	0.65	2.85	11.89	CPU
WNNM	203.1	773.2	2 536.4	CPU
EPLL	25.4	45.5	422.1	CPU
MLP	1.42	5.51	19.4	CPU
CSF	2.11 / —	5.67 / 0.92	40.8 / 1.72	CPU / GPU
TNRD	0.45 / 0.010	1.33 / 0.032	4.61 / 0.116	CPU / GPU
DnCNN-S	0.74 / 0.014	3.41 / 0.051	12.1 / 0.200	CPU / GPU
DnCNN-B	0.90 / 0.016	4.11 / 0.060	14.1 / 0.235	CPU / GPU

State the asymmetry before you are asked. The paper itself warns that *since GPU run time varies greatly with respect to GPU and GPU-accelerated library, it is hard to make a fair comparison* between CSF, TNRD and DnCNN — the authors simply **copied the CSF and TNRD GPU times from the original papers**. And BM3D's 2.85 s is a CPU figure being set beside a GPU figure. The honest headline is: **DnCNN-B is 12 900× faster than WNNM at 512×512 (773.2 s CPU → 0.060 s GPU)**, with the hardware difference named.

DnCNN17 Table IV and its caption

One model, three tasks — 'beyond a Gaussian denoiser'

The conceptual leap that gives the paper its name. Reinterpret v , and the identical network, identical loss and identical training procedure solve two problems that are not denoising.

The reinterpretation

- **Denoising:** v is AWGN, $\sigma \in [0, 55]$.
- **Super-resolution:** if v is *the difference between a ground-truth high-resolution image and the bicubic upsampling of its low-resolution version*, the same degradation model becomes single-image SR. Scales $\times 2$, $\times 3$, $\times 4$.
- **JPEG deblocking:** if v is *the difference between the original image and its compressed version*, the model becomes JPEG deblocking. Quality factors 5–99.
- The paper's conclusion: SISR and JPEG deblocking *are special cases of a 'general' image denoising problem* — even though their v is very different from AWGN.

The theoretical permission, and the results

- It is not asserted but derived: from the connection with one-stage TNRD, the residual estimate remains valid whenever *the derivative of the data-fidelity term vanishes at zero*, a condition holding for *many types of noise distributions, e.g. the generalized Gaussian distribution*. The authors state it is natural to assume it also holds for SISR and JPEG noise.
- **Denoising, BSD68 $\sigma=25$:** BM3D 28.57, TNRD 28.92 → **DnCNN-3 29.02**.
- **SR, Set5 $\times 3$:** TNRD 33.18, VDSR 33.67 → **DnCNN-3 33.75**. Set14 $\times 3$: TNRD 29.43, VDSR 29.77 → **29.81**.
- **JPEG, Classic5 QF10:** AR-CNN 29.03, TNRD 29.28 → **DnCNN-3 29.40**. LIVE1 QF20: AR-CNN 31.29, TNRD 31.46 → **31.59**.
- **Reproduce the hedge.** The paper says *to the best of our knowledge* none of the existing methods had been reported for handling these three tasks with only a single model. Overstating priority is exactly what a defence punishes.

What DnCNN settled, and the bill it left behind

DnCNN closed bottleneck 1 completely. It removed test-time optimisation, removed hand-tuned constants, removed the per-noise-level model, and made denoising real-time. In doing so it created a new dependency that the next era spent seven years attacking.

SETTLED

Test-time cost

One forward pass. 0.060 s for a 512×512 image. WNNM's 773 s is gone and never comes back — every method after 2017 is a feed-forward network.^[1]

THE BILL

Paired clean data

The loss is $\|\mathcal{R}(y) - (y - x)\|^2$. It requires x . DnCNN's^[1] 400 clean images plus synthetic noise are cheap for natural photographs and **impossible** for fluorescence microscopy, cryo-electron microscopy, MRI, or astronomy, where a clean ground truth does not exist and cannot be acquired.^[2]

SETTLED

Blind operation

DnCNN-B^[1] needs no σ estimate. The workaround of estimating noise first and loading a matching model — which made the result hostage to the estimate — is obsolete.

THE BILL

The local receptive field

35×35 pixels. Barbara's scarf, −0.71 dB against BM3D.^[1] The non-local self-similarity prior that BM3D and WNNM exploited for free is simply unavailable to a stack of 3×3 convolutions.

The 2018 fork. These two debts were attacked independently and almost simultaneously, by different communities, with different methods — and they never merged. Parts IV and V follow one branch each.

^[1] reading, grounded in DnCNN17 §III-A Eq. (1), Table I and Table III ^[2] N2V19 §1 on the unavailability of ground truth

IV

The Data Branch

— PART IV · 2018 – 2025 · BRANCH ONE

— In this part

- 01 Noise2Noise — clean targets are unnecessary
- 02 Noise2Void — a second noisy copy is unnecessary
- 03 ZS-N2N — a training set is unnecessary
- 04 P2N — discarding information is unnecessary
- 05 The through-line: the network never moved, the target did

Seven years spent removing the clean image from the training signal, one requirement at a time — while the network stayed almost exactly where DnCNN left it.

The statistical argument that removed the clean image

Lehtinen, Munkberg, Hasselgren, Laine, Karras, Aittala and Aila, ICML 2018 (NVIDIA). The paper applies *basic statistical reasoning* and reaches a conclusion that sounds impossible: you can learn to restore images by looking only at corrupted examples.

$$\arg \min_z \mathbb{E}_y \{L(z, y)\} \quad \text{with} \quad L(z, y) = (z - y)^2 \quad \implies \quad z = \mathbb{E}_y \{y\}$$

the L2 minimiser of a set of observations is their arithmetic mean. The L1 minimiser is the median.

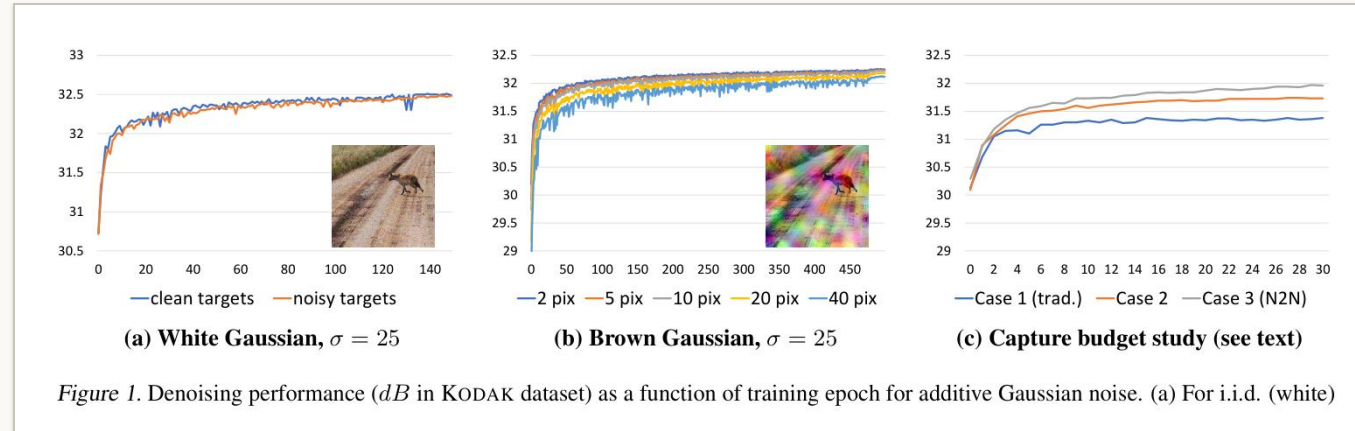
$$\arg \min_{\theta} \sum_i L(f_{\theta}(\hat{x}_i), \hat{y}_i) \quad \text{subject to} \quad \mathbb{E}\{\hat{y}_i \mid \hat{x}_i\} = y_i$$

Equation (6) — both inputs and targets are drawn from a corrupted distribution, conditioned on the unobserved clean target

- 01 **The argument.** On expectation the estimate is unchanged if targets are replaced by random numbers with matching expectations — so targets may be corrupted by zero-mean noise **without changing what the network learns**.
- 02 **Given infinite data the solution equals clean-target training.** For finite data the extra variance shrinks with the sample count rather than persisting.
- 03 **Not required:** no likelihood model of the corruption, no density model of the clean image manifold.
- 04 **Why it is physically natural.** A long, noise-free exposure is the average of short noisy ones — so two short exposures of one scene are exactly the pair the theorem asks for.
- 05 **The loss chooses the estimator.** L2 recovers the mean; L1 the median, tolerating up to 50% outliers.

Noisy targets cost essentially nothing

PSNR on three test sets for three noise types, from Table 1. In each pair, *clean* is conventional supervised training and *noisy* is the same network trained on corrupted targets only.



Denoising PSNR on Kodak as training progresses: networks trained on noisy targets (orange) track clean-target training (blue) almost exactly, for white and correlated Gaussian noise alike. [N2N18 · Fig. 1]

TEST SET	GAUSSIAN $\Sigma=25$ CLEAN	NOISY	BM3D	POISSON $\Lambda=30$ CLEAN	NOISY	BERNOULLI $P=0.5$ CLEAN	NOISY
Kodak	32.50	32.48	31.82	31.52	31.50	33.01	33.17
BSD300	31.07	31.06	30.34	30.18	30.16	31.04	31.16
Set14	31.31	31.28	30.50	30.07	30.06	31.51	31.72
Average	31.63	31.61	30.89	30.59	30.57	31.85	32.02

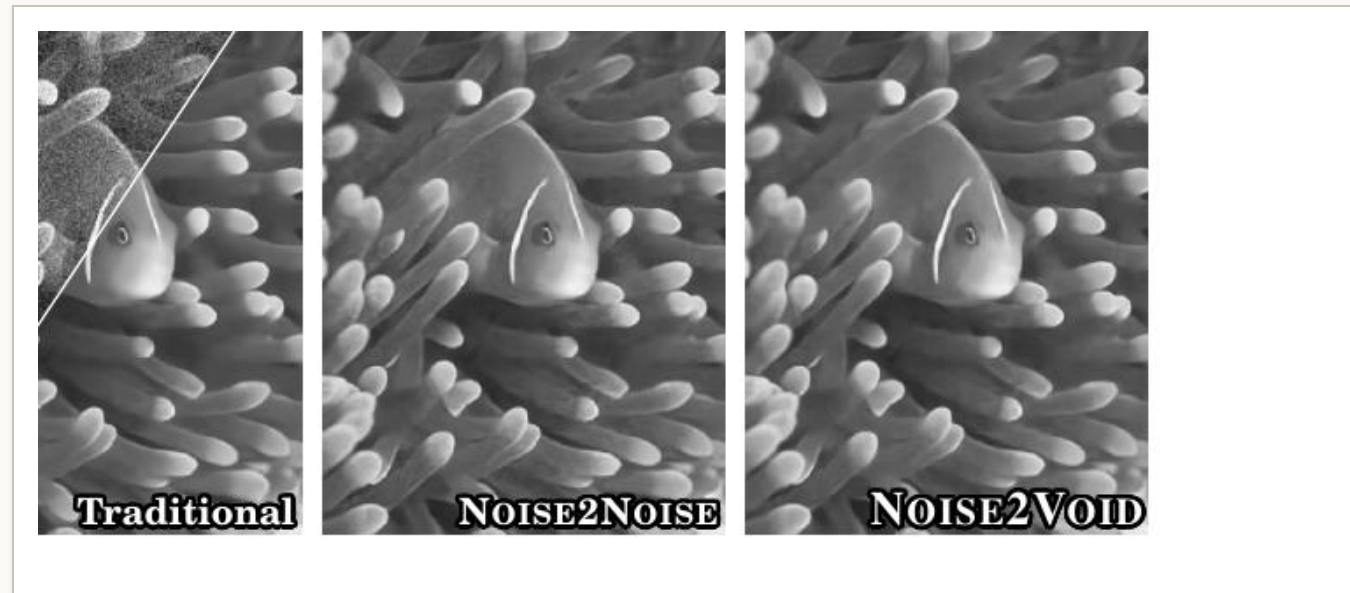
Noisy targets cost essentially nothing

The gap between clean-target and noisy-target training is 0.02 dB on Gaussian noise and 0.02 dB on Poisson. For Bernoulli noise the noisy-target model is *better*. Setting the deep-learning requirement for clean data aside costs about one fiftieth of the margin DnCNN gained over BM3D. Training used 256×256 crops from the 50 000 ImageNet validation images with σ randomised per example in $[0, 50]$ — i.e. blind denoising.

N2N18 Table 1, §3.1 — values as printed

Blind-spot networks — removing the second noisy copy

Krull, Buchholz and Jug, CVPR 2019 (MPI-CBG Dresden). Noise2Noise still needs *two independent noisy observations of the same scene*. For a fixed specimen under a microscope that is often two acquisitions; for a moving world, or a single archival image, it is nothing. N2V removes it.



The three training schemes side by side: conventional noisy/clean pairs, Noise2Noise's noisy/noisy pairs, and Noise2Void's single noisy image with a blind spot.
[N2V19 · Fig. 1]

Blind-spot networks — removing the second noisy copy

$$p(s, n) = p(s) p(n \mid s), \quad p(n \mid s) = \prod_i p(n_i \mid s_i), \quad \mathbb{E}[n_i] = 0 \implies \mathbb{E}[x_i] = s_i$$

the two assumptions: (i) the signal is **not** pixel-wise independent, $p(s_i | s_j) \neq p(s_i)$; (ii) the noise **is** conditionally pixel-wise independent given the signal, and zero-mean

- 01 **The problem.** Same noisy image as input and target degenerates — the identity has zero loss and does nothing.
- 02 **The fix: a blind spot.** The prediction for pixel i uses its neighbourhood *except pixel i itself*.
- 03 **Why it cannot learn the identity.** Given conditional pixel-wise independence, neighbours *carry no information about n_i* . The signal does have dependencies, so it can still be estimated. Signal learnable, noise not.
- 04 **The implementation is a trick.** N2V *replaces the centre value of each patch with a randomly selected value from the surrounding area*, and scores the loss only there.
- 05 **Efficiency:** 64 pixels masked per 64×64 patch — 1.6% — rather than one.

What self-supervision costs, measured

Four training regimes, one network family, one dataset: BSD68 with $\sigma = 25$ Gaussian noise, trained on 400 grayscale images at 180×180 — deliberately the same protocol DnCNN used. The ladder shows exactly what each removed requirement costs.

29.06

TRADITIONAL · CLEAN TARGETS

Supervised U-Net, depth 2, 96 initial feature maps. The ceiling.

28.86

NOISE2NOISE · NOISY PAIRS

−0.20 dB for giving up the clean image.

28.59

BM3D · NO TRAINING AT ALL

The classical zero-shot baseline, seventeen years old.

27.71

NOISE2VOID · SINGLE NOISY IMAGES

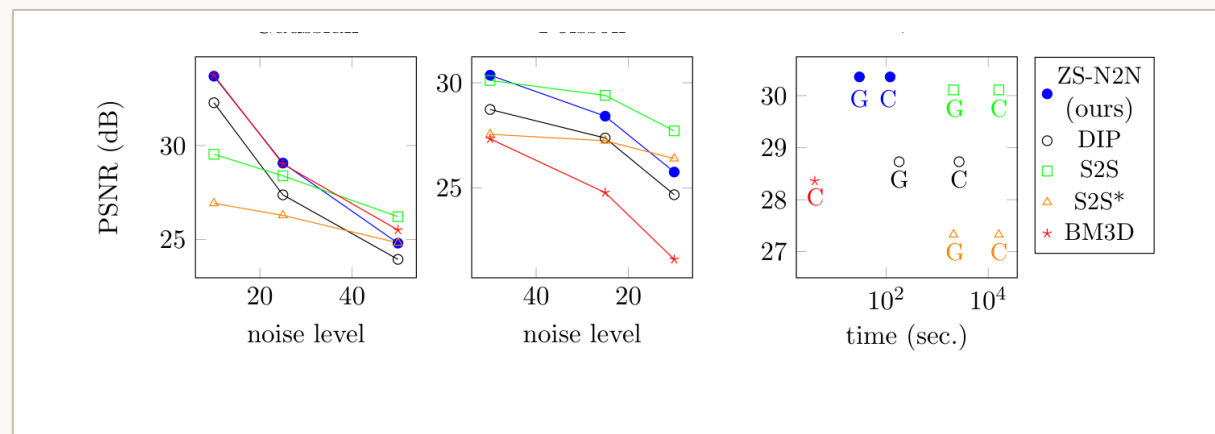
−1.35 dB from supervised, and 0.88 dB **below** BM3D in N2V's own table.

- 01 Read the ladder honestly, and say it before you are asked.** On natural images with plentiful clean data, N2V is *worse than a 2007 algorithm*. The paper does not hide this — it states that its performance *only drops moderately* and notes it still outperforms BM3D on its other benchmarks.
- 02 So why does N2V matter?** Because the comparison above is the one case where the alternatives exist. The paper's real argument is the three biomedical datasets where they do not: cryo-TEM, and two Cell Tracking Challenge sets. For CTC-MSD and CTC-N2DH *only single noisy images exist*, so neither traditional nor N2N training is applicable — and the ground-truth column in the paper's figure is literally captioned *does not exist*.
- 03 The correct framing for the defence.** Self-supervision is not a quality technique. It is an *availability* technique. It trades PSNR for applicability, and in the domains it targets, the alternative is not a lower score — it is no denoiser at all.

N2V19 §4.1, Fig. 4 — PSNR values as printed

Zero-shot — removing the training set entirely

Mansour and Heckel, CVPR 2023 (TU Munich / Rice). N2V still needs a *body of noisy data to train on*. ZS-N2N needs one image: the one you want to denoise. It has no training set, no pretraining, and about 20 000 parameters.



ZS-N2N's own benchmark: PSNR against noise level for Gaussian and Poisson noise, and quality against wall-clock time on CPU and GPU, against DIP, S2S and BM3D. [ZS-N2N23 · Fig. 1]

$$D_1 = y \circledast k_1, \quad k_1 = \begin{pmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{pmatrix} \quad D_2 = y \circledast k_2, \quad k_2 = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \quad (\text{stride } 2)$$

the **pair downsampler** — each 2×2 block splits into its diagonal and anti-diagonal average

Zero-shot — removing the training set entirely

$$L_{\text{res}} = \frac{1}{2} \left(\|D_1 - f_{\theta}(D_1) - D_2\|^2 + \|D_2 - f_{\theta}(D_2) - D_1\|^2 \right)$$

symmetric **residual** loss — f_{θ} predicts the noise, exactly as DnCNN does, six years later

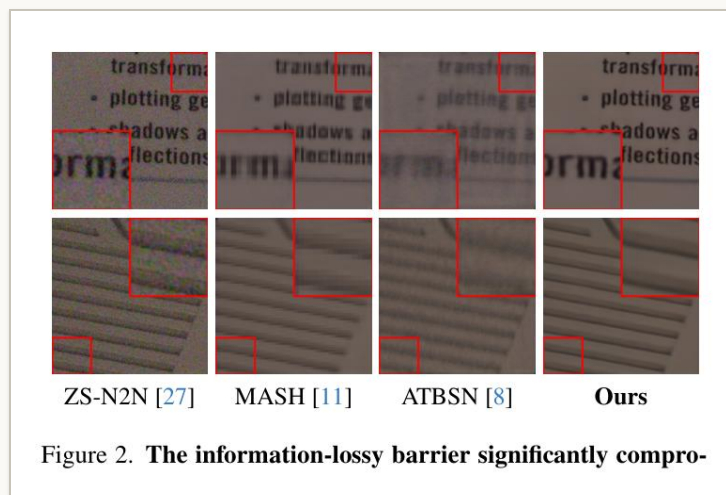
$$L_{\text{cons}} = \frac{1}{2} \left(\|D_1 - f_{\theta}(D_1) - D_1(y - f_{\theta}(y))\|^2 + \|D_2 - f_{\theta}(D_2) - D_2(y - f_{\theta}(y))\|^2 \right)$$

consistency — denoise-then-downsample must agree with downsample-then-denoise

- 01 **Minimise $L = L_{\text{res}} + L_{\text{cons}}$ on that one image**, then estimate $\hat{x} = y - f_{\theta}(y)$. Only θ is optimised; D_1 and D_2 are fixed convolutions.
- 02 **Why the pair is valid.** Nearby clean pixels are correlated while noise is not, so the pair has *similar signal but independent noise*.
- 03 **Why consistency matters.** The residual loss only sees half resolution; the consistency term is the only one that sees the full image, and it removes the need for early stopping.
- 04 **The network:** two 3×3 operators and one 1×1 — **≈ 20 K parameters**, a minute on CPU. A U-Net overfits and does *much worse*. For scale, Self2Self needs *1.2 hours for one 256×256 image on a GPU*.

Positive2Negative — removing the information loss

Li, Wang, Xu, Zhu, Lu and Huang, CVPR 2025. Every self-supervised method so far pays for its training signal by *throwing information away*: N2V blinds a pixel, ZS-N2N halves the resolution. P2N's claim is that neither sacrifice is necessary.



The information-lossy barrier, made visible: ZS-N2N, MASH and ATBSN all lose fine text and fabric structure that P2N's renoise-and-agree supervision preserves.
[P2N25 · Fig. 2]

Positive2Negative — removing the information loss

The two mechanisms^[1]

- **RDC — Renoised Data Construction.** Denoise the image once, estimate the noise, then *add that estimated noise back* with a plus and with a minus, producing y^+ and y^- . Both retain the full resolution and every pixel of the original.
- **DCS — Denoised Consistency Supervision.** Denoise y^+ and y^- and train the two results to agree. The supervision signal is the disagreement between two renoised views of the same image.
- Nothing is masked. Nothing is downsampled. The barrier the title names — *the information-lossy barrier in self-supervised single image denoising* — is the masking-and-downsampling tax that N2V and ZS-N2N both pay.

Why this is a *supervision* change, not a loss change

- This distinction matters and it is easy to get wrong. P2N does not merely swap one loss function for another.
- **It changes what goes into the network.** The inputs y^+ and y^- are constructed from the model's own output — they do not exist until the model has run once.
- **It changes how many forward passes there are.** Three, not one: denoise y , then denoise y^+ , then denoise y^- .^[2]
- **It changes which pair is scored.** The comparison is output-against-output, not output-against-target.
- A loss function scores a fixed prediction against a fixed target. P2N rewrites the whole training signal — what is shown, how many times, and what is compared.

And the architecture? Unchanged. The network class in the official repository is literally named `UNet_n2n_un` and matches the 2018 Noise2Noise channel widths exactly. P2N+ adds one optional module, Symmetric Prior Injection. Seven years of progress on this branch, and the network is the same one. [\[measured — P2N-plus, commit 8a117db\]](#)

^[1] P2N25 (CVPR 2025, pp. 17924–17934; arXiv:2412.16460) ^[2] repository `github.com/Li-Tong-621/P2N-plus` [measured]`

The network never moved. The target did.

Four papers, seven years, one branch. Read them as a single sequence and each one removes exactly one requirement its predecessor could not drop — while the backbone stays a U-Net of roughly the 2018 shape throughout.

2018

Noise2Noise[1]

Target: a second noisy photograph of the same scene. **Requirement removed:** the clean image. **Cost:** you must capture the scene twice, which rules out anything that moves and every archival image.

2019

Noise2Void[2]

Target: the masked pixel's own original value. **Requirement removed:** the second copy. **Cost:** 1.6% of pixels are blinded per patch, and the noise must be spatially independent — which excludes correlated real noise.

2023

ZS-N2N[3]

Target: the other diagonal of the same 2×2 block. **Requirement removed:** the training set. **Cost:** half the spatial resolution during training, plus a per-image optimisation of 1–2 K steps.

2025

P2N[4]

Target: the model's own output on a renoised copy. **Requirement removed:** the information loss itself. **Cost:** three forward passes per step, and per-image test-time optimisation.

The arc in one sentence: the prior moved out of the algorithm and into the weights in 2017 — and then, on this branch, partly back *out* of the weights and into a per-image optimisation that needs no clean data at all. The classical era's test-time optimisation has returned, but the thing being optimised is now a network, not a pixel grid.

[1] N2N18 [2] N2V19 [3] ZS-N2N23 [4] P2N25 — each paper's own stated requirement



The Architecture Branch

— PART V · 2021 – 2024 · BRANCH TWO

— In this part

- 01 Bottleneck 3, stated with evidence
- 02 Attention, and why it does not fit
- 03 Restormer — attention across channels
- 04 NAFNet — is any of this necessary?
- 05 SCUNet — fix the data, not the network
- 06 MambaIR — linear-time global context
- 07 Three competing answers

The other half of the fork. Same supervision as DnCNN, same paired data, same loss family — and a complete replacement of the 3×3 convolution.

Convolution cannot see far enough — and here is the receipt

The third bottleneck is the only one visible in DnCNN's own results table, three years before anyone could address it. Two independent pieces of evidence, taken from two papers five years apart, say the same thing.

Evidence 1 — Barbara, 2017^[1]

- DnCNN-S loses to BM3D by **0.71 dB** and to WNNM by **1.24 dB** on Barbara at $\sigma = 25$.
- The paper's own explanation: DnCNN fails *on exactly the two images dominated by repetitive structures*, because those images *meet well with the non-local similarity prior*.
- DnCNN's receptive field is 35×35 . WNNM's effective patch size is **361×361** — ten times wider in each direction.
- The scarf's stripes have near-identical twins two hundred pixels away. DnCNN cannot see them. There is no amount of training that fixes a structural blindness.

Evidence 2 — Urban100, 2022^[2]

- Urban100 is 100 photographs of buildings: windows, balconies and railings repeating identically across the whole frame. It is Barbara's scarf at dataset scale.
- Grayscale $\sigma = 50$: **DnCNN 26.35 → Restormer 28.33**. A gap of **1.98 dB**.
- On BSD68 — ordinary natural images, little repetition — the same two models are 26.23 and 29.62: a gap of **0.39 dB**.
- **The gap is five times larger on the dataset made of repetition.** That ratio is the cleanest single measurement of bottleneck 3 in the literature, and it is computed from one table.

Restate the bottleneck. A 3×3 convolution aggregates a 3×3 neighbourhood. Stacking d of them without pooling reaches $(2d+1) \times (2d+1)$ — growth is *linear in depth* and therefore ruinously expensive. Reaching 361×361 the DnCNN way needs 180 layers. The architecture branch is the search for an operator whose reach is not bought by depth.

^[1] DnCNN17 Table III, Table I, §IV-C ^[2] Restormer22 Table 4 — Urban100 and BSD68 columns

Self-attention — the obvious fix, and why it does not fit

Attention was invented for sequences and lets any element consult any other in a single operation. That is exactly the property bottleneck 3 needs. Its cost is exactly what makes it unusable on images.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Q, K, V — query, key and value projections of the input · the softmax produces one weight per pair of elements

$$\text{cost} = O(W^2 H^2) \quad \text{for a } W \times H \text{ image}$$

the key–query dot-product interaction grows **quadratically with the spatial resolution**

- 01 **The numbers make the problem concrete.** A 256×256 image has 65 536 spatial positions, so the attention matrix holds $\approx 4.3 \times 10^9$ entries. A ViT at 224×224 with 16×16 patches has 196 tokens. Restoration and classification are not the same problem.[\[1\]](#)
- 02 **Restormer[\[2\]](#) states the constraint directly:** *it is infeasible to apply SA on most image restoration tasks that often involve high-resolution images.*
- 03 **Fix 1 — restrict the window.** SwinIR[\[3\]](#) attends inside fixed local windows, shifting them between layers. Restormer's[\[2\]](#) objection is structural: *restricting the spatial extent of SA is contradictory to the goal of capturing the true long-range pixel relationships.*

[\[1\]](#) Vaswani et al., NeurIPS 2017 (standard formulation) [\[2\]](#) Restormer22 §1, §3.1 [\[3\]](#) SCUNet23 §1 on SwinIR

Self-attention — the obvious fix, and why it does not fit

- 04 **Fix 2 — change the axis.** If 65 536 positions are unaffordable, attend over the *channels* instead. There are 48.
- 05 **Fix 3 — abandon attention.** NAFNet argues the gains were never attention's; MambaIR replaces it with a recurrence linear in sequence length.

[1] Vaswani et al., *NeurIPS 2017 (standard formulation)* [2] Restormer22 §1, §3.1 [3] SCUNet23 §1 on SwinIR

MDTA — self-attention across channels, not pixels

Zamir, Arora, Khan, Hayat, Khan and Yang, CVPR 2022. The key design: apply self-attention across the feature dimension rather than the spatial dimension, so the attention map is $C \times C$ instead of $HW \times HW$.

$$Q = W_d^Q W_p^Q Y, \quad K = W_d^K W_p^K Y, \quad V = W_d^V W_p^V Y$$

W^p — 1×1 point-wise convolution aggregating cross-channel context · W^d — 3×3 depth-wise convolution encoding channel-wise spatial context

$$\text{Attention}(\hat{Q}, \hat{K}, \hat{V}) = \hat{V} \cdot \text{softmax}\left(\frac{\hat{K}\hat{Q}^\top}{\alpha}\right)$$

Q, K, V reshaped so the dot product yields a $C \times C$ transposed-attention map — cross-covariance across channels, not pairwise pixel interactions

- 01 **Linear complexity.** The attention matrix is $C/k \times C/k$ per head, independent of H and W . At level 1, $C = 48$ — a 48×48 matrix whatever the image size.
- 02 **What the global context is.** Rather than pairwise pixel interactions, MDTA computes *cross-covariance across feature channels*. Each channel is a global descriptor, so weighting them by mutual covariance encodes image-wide relationships.
- 03 **The depth-wise convolutions are not decoration.** The paper calls local context mixing *an important feature*; removing it drops Urban100 $\sigma=50$ from 29.43 to 29.28.
- 04 **Two claims:** it brings *the complimentary strength of convolution operation* into the pipeline, and global relationships are *implicitly modeled*.

GDFN and progressive learning

The second half of the Transformer block, and a training strategy that is worth as much as an architectural change.

$$\text{Gate}(X) = \phi(W_d^1 W_p^1 \text{LN}(X)) \odot (W_d^2 W_p^2 \text{LN}(X))$$

the gated-Dconv feed-forward network: two parallel projections, one passed through GELU (ϕ), multiplied element-wise · expansion factor $\gamma = 2.66$

- 01 **The role of the gate.** A standard feed-forward network is two linear layers with a nonlinearity between them. GDFN replaces the first with a *gating mechanism* — the element-wise product of two projections, one GELU-activated. The gate *controls which complementary features should flow forward*, allowing later layers to focus on more refined attributes.
- 02 **Ablation value.** Gating alone gives +0.12 dB over conventional FN; adding depth-wise convolution to it gives **+0.26 dB** at $\sigma = 50$ on Urban100.
- 03 **Progressive learning.** The network trains on *small patches and large batches in early epochs, and gradually large image patches and small batches in later epochs*. Concretely: begin at 128^2 with batch 64, then step through $(160^2, 40)$, $(192^2, 32)$, $(256^2, 16)$, $(320^2, 8)$, $(384^2, 8)$ at iterations 92 K, 156 K, 204 K, 240 K, 276 K.
- 04 **Why it helps.** It is curriculum learning — start with an easier task and move to a harder one where preserving fine structure and texture is required. Because a model trained on mixed-size patches sees more context distributions, it performs better *at test time on images of different resolutions*, which is the normal case in restoration. Batch size falls as patch size rises so the time per optimisation step stays roughly constant.
- 05 **Full training recipe.** 4-level encoder–decoder; blocks [4, 6, 6, 8]; heads [1, 2, 4, 8]; channels [48, 96, 192, 384]; 4 refinement blocks; AdamW (β_1 0.9, β_2 0.999, weight decay $1e^{-4}$); **L1 loss**; 300 K iterations; LR $3e^{-4} \rightarrow 1e^{-6}$ by cosine annealing; horizontal and vertical flips only.

The ablation that isolates every claim

Colour denoising models trained on 128×128 patches for 100 K iterations, tested on Urban100. FLOPs and inference time at 256×256. This is the table to point at when asked whether the components actually do anything.

NETWORK COMPONENT	FLOPS (B)	PARAMS (M)	$\Sigma=15$	$\Sigma=25$	$\Sigma=50$
(a) Baseline — UNet with Resblocks	83.4	24.53	34.42	32.18	29.11
(b) MTA + FN	83.7	24.84	34.66	32.39	29.28
(c) MDTA + FN	85.3	25.02	34.72	32.48	29.43
(e) MTA + DFN	85.8	25.08	34.68	32.45	29.42
(f) MTA + GDFN	86.2	25.12	34.77	32.56	29.54
(g) MDTA + GDFN	87.7	25.31	34.82	32.61	29.62
+ Concat (no 1×1)	110	25.65	—	—	29.66
+ Refinement stage	141	26.12	—	—	29.71

Total contribution of the Transformer block: +0.51 dB over the baseline at $\sigma = 50$, for a 5% increase in FLOPs. MDTA alone contributes +0.32 dB; GDFN alone contributes +0.26 dB over standard FN. Note also the deeper-versus-wider finding: at matched budget a **deep-narrow** model is more accurate, while a wide-shallow one runs faster through better parallelisation. Restormer ships deep-narrow.

Restormer22 Table 7, Table 8, Table 10 — values as printed

Gaussian denoising — and the shape of the win

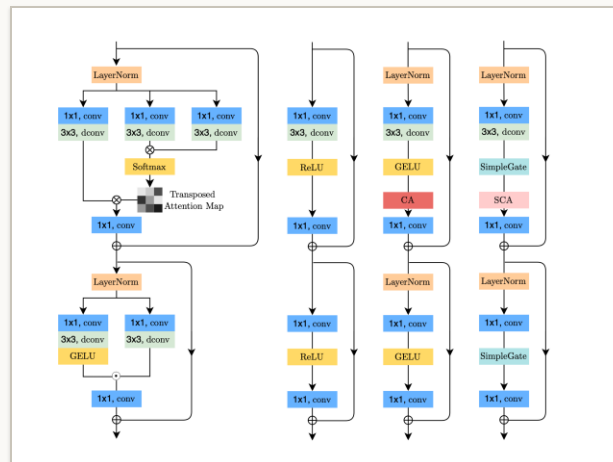
Grayscale PSNR from Table 4. The top block is a *single model handling all noise levels*; the bottom block is a *separate model per noise level*. Restormer leads both.

METHOD	SET12 $\Sigma=15$	$\Sigma=25$	$\Sigma=50$	BSD68 $\Sigma=15$	$\Sigma=25$	$\Sigma=50$	URBAN100 $\Sigma=15$	$\Sigma=25$	$\Sigma=50$
DnCNN (2017)	32.67	30.35	27.18	31.62	29.16	26.23	32.28	29.80	26.35
FFDNet	32.75	30.43	27.32	31.63	29.19	26.29	32.40	29.90	26.50
IRCNN	32.76	30.37	27.12	31.63	29.15	26.19	32.46	29.80	26.22
DRUNet	33.25	30.94	27.90	31.91	29.48	26.59	33.44	31.11	27.96
Restormer (single)	33.35	31.04	28.01	31.95	29.51	26.62	33.67	31.39	28.33
NLRN	33.16	30.80	27.64	31.88	29.41	26.47	33.45	30.94	27.49
SwinIR	33.36	31.01	27.91	31.97	29.50	26.58	33.70	31.30	27.98
Restormer (per-σ)	33.42	31.08	28.00	31.96	29.52	26.62	33.79	31.46	28.29

Read across the row, not down the column. From DnCNN to Restormer, BSD68 $\sigma=50$ moves 26.23 $\rightarrow$ 26.62 (**+0.39 dB**) while Urban100 $\sigma=50$ moves 26.35 $\rightarrow$ 28.33 (**+1.98 dB**). The architecture branch did not make denoising uniformly better. It made denoising better *specifically where non-locality matters* — which is exactly what it set out to do. Efficiency note: Restormer has **3.14× fewer FLOPs** and **runs 13× faster** than SwinIR, and unlike DRUNet it requires no noise-level map as input.

NAFNet — the paper that asked whether any of this is necessary

Chen, Chu, Zhang and Sun, ECCV 2022 (MEGVII). While everyone else added mechanisms, this paper removed them, and reached a higher number. Its two derivations are the most elegant reasoning in the modern half of this deck.



The lineage the paper derives in one figure: the Restormer transformer block simplified step by step — ReLU in place of softmax attention, then GELU with channel attention, then NAFNet's SimpleGate and SCA. [NAFNet22 · Fig. 3]

$$\text{GLU}(X, f, g, \sigma) = f(X) \odot \sigma(g(X))$$

the gated linear unit, and GELU — Φ is the standard normal CDF

$$\text{GELU}(x) = x \Phi(x)$$

NAFNet — the paper that asked whether any of this is necessary

$\Rightarrow$ GELU is a special case of GLU, with $f, g = \text{id}$ and $\sigma = \Phi$

which suggests GLU is a generalisation of activation functions — and that it might replace them

$\text{SimpleGate}(X, Y) = X \odot Y$

split the feature map in half along the channel axis and multiply the halves. That is the entire activation function.

- 01 **The key observation.** GLU itself contains nonlinearity and does not depend on σ — $f(X) \odot g(X)$ is nonlinear because a product of two linear maps is not linear. So the gate alone supplies it.
- 02 **Compare implementations.** GELU needs $0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$. SimpleGate is one element-wise multiplication — *That's all.*
- 03 **Applied again to channel attention,** $\text{CA}(X) = X * \Psi(X)$ reduces to **SCA(X) = X * W pool(X)**: one pooling, one 1×1 , no Sigmoid, no ReLU.
- 04 **The result.** No activation functions anywhere — *the first work demonstrating that the nonlinear activation functions may not be necessary for SOTA computer vision methods.*

The ablation ladder — every rung is a controlled experiment

Starting from PlainNet — convolution, ReLU, shortcut, in a single-stage U-Net — each row changes exactly one thing. All at a ≈ 16 GMAC budget, 256 $\times$ 256 patches, batch 32, 200 K iterations, Adam, PSNR loss.

STEP	CHANGE MADE	SIDD (DENOISING)	GOPRO (DEBLURRING)
0	PlainNet — conv + ReLU + shortcut	39.29	28.51
1	+ LayerNorm (enables 10 $\times$ learning rate)	39.73 (+0.44)	31.90 (+3.39)
2	ReLU $\rightarrow$ GELU	39.71 (−0.02)	32.11 (+0.21)
3	+ Channel Attention	39.85 (+0.14)	32.35 (+0.24)
4	GELU $\rightarrow$ SimpleGate	39.93 (+0.08)	32.76 (+0.41)
5	CA $\rightarrow$ SCA	39.96 (+0.03)	32.85 (+0.09)
—	Final NAFNet at full capacity	40.30	33.69

Three readings. (a) The largest single gain is **LayerNorm**, worth +0.44 dB on denoising and +3.39 dB on deblurring — not because normalisation is magic but because it *stabilises training at a ten-times larger learning rate*. The architecture did not improve; the optimisation did. (b) Both simplifications **improved** results — SimpleGate beats GELU, SCA beats CA. (c) The headline: **40.30 dB on SIDD, exceeding the previous state of the art by 0.28 dB with less than half its computational cost**, and 36.69 dB on GoPro, exceeding the previous best by 0.38 dB with **8.4%** of its cost.

SCUNet — fix the training data, not only the network

Zhang, Li, Liang, Cao, Zhang, Tang, Fan, Timofte and Van Gool, *Machine Intelligence Research* 20(6), 2023 (ETH Zürich). Same first author as DnCNN, six years later, attacking the assumption DnCNN made rather than the architecture DnCNN built.

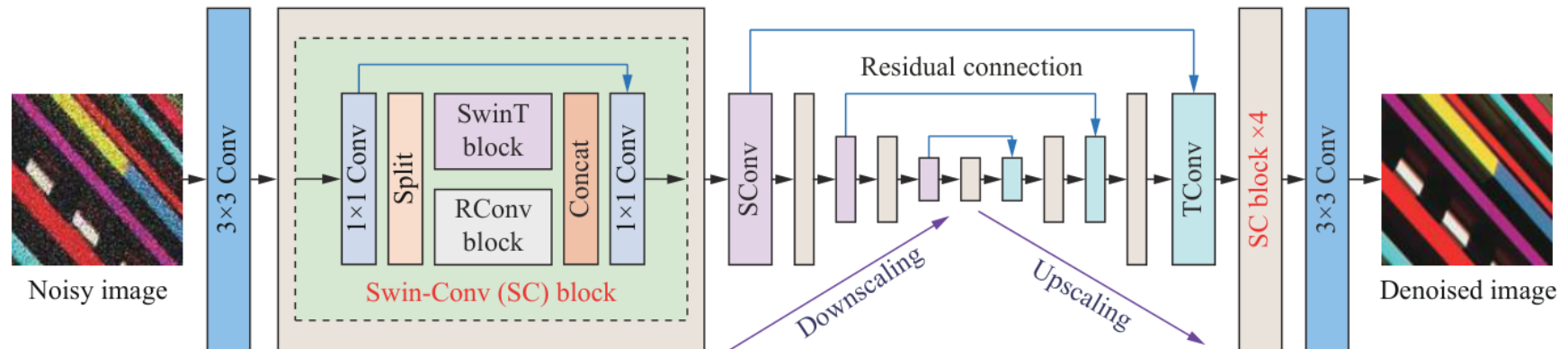


Fig. 1 The architecture of the proposed Swin-Conv-UNet (SCUNet) denoising network. SCUNet exploits the swin-conv (SC) block as

The Swin-Conv-UNet pipeline: a noisy image passes through strided SC blocks (Swin Transformer + residual convolution in parallel) on the way down, and transposed-convolution SC blocks on the way back up, with a global residual connection. [SCUNet23 · Fig. 1]

SCUNet — fix the training data, not only the network

Half one — the swin-conv block^[1]

- The motivation is complementarity: *different methods for image denoising have complementary image prior modeling ability and can be incorporated to boost the performance.*
- **The block.** A 1×1 convolution, then split the feature map in two. One half goes to a **Swin Transformer block** (non-local), the other to a **residual convolutional block** (local). Concatenate, 1×1 , residual connection.
- Plug it into a standard UNet backbone. Local and non-local priors run **in parallel** rather than one replacing the other.
- **9.663 M parameters** — a third of Restormer's, a twelfth of NAFNet's. ^[2]

Half two — the degradation model^[1]

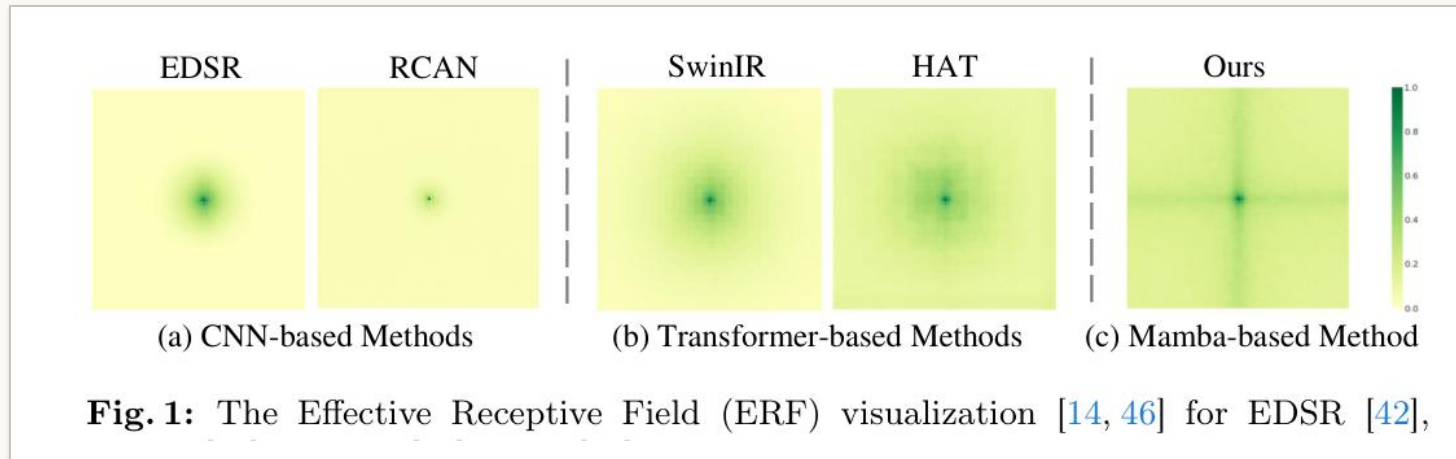
- **The premise, bluntly:** a model trained by AWGN *performs poorly for most of real images due to noise assumption mismatch*. Nevertheless AWGN removal is fair to test the effectiveness of different network architecture designs.
- **So synthesise realistic noise.** The pipeline samples Gaussian (3-D generalised, with a 3×3 covariance matrix for R/G/B correlation), **Poisson**, **speckle**, **JPEG**, and **camera sensor noise**.
- **Sensor noise goes through a real ISP** — invert a clean image to raw, add read and shot noise there, then run demosaicing, white balance, tone mapping and gamma forward.
- **Plus resizing, random shuffling and double degradation**, because *upsampling would lead AWGN to be spatially correlated* and real images may be resized or JPEG compressed repeatedly.

The lesson worth stating. Every other paper in Part V competes on the architecture. SCUNet's four contributions include *a hand-designed noise synthesis model, which can be used to train a general-purpose blind image denoising model*. The distribution you train on is a design surface, and it is the one that most directly attacks the AWGN assumption named on slide 7.

^[1] SCUNet23 abstract, §1, §3.1, §3.2, Fig. 1 ^[2] parameter count [measured], commit 52e440a

State-space models — a third answer to bottleneck 3

Guo, Li, Dai, Ouyang, Ren and Xia, ECCV 2024 (Tsinghua Shenzhen / Shenzhen University / ByteDance). If attention is quadratic and windowed attention surrenders globality, borrow a mechanism from control theory that is global *and* linear.



Effective receptive field, visualised: CNN backbones (EDSR, RCAN) stay tightly local, Transformers (SwinIR, HAT) spread further, and the Mamba-based backbone activates the widest, most global field. [MambaIR24 · Fig. 1]

$$h'(t) = A h(t) + B x(t), \quad y(t) = C h(t) + D x(t)$$

a continuous linear time-invariant system — the S4 formulation. N is the state size; $A \in \mathbb{R}^{N \times N}$, $B \in \mathbb{R}^{N \times 1}$, $C \in \mathbb{R}^{1 \times N}$, $D \in \mathbb{R}$

State-space models — a third answer to bottleneck 3

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1} (\exp(A) - I) \cdot \Delta B$$

Δ is the timescale parameter converting the continuous system to a discrete one

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k, \quad y_k = Ch_k + Dx_k \quad \Longleftrightarrow \quad \bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{L-1}\bar{B}), \quad y = x \circledast \bar{K}$$

the same model in **RNN form** (sequential, $O(L)$ memory of the whole past) and in **CNN form** (a single long convolution, parallelisable)

- 01 **Why the dual form is the trick.** The recurrence gives an unbounded receptive field; the convolutional form allows parallel training. Mamba keeps both.
- 02 **What Mamba adds over S4.** B, C and Δ become **input-dependent**, giving content-adaptive behaviour rather than one fixed linear system.
- 03 **Complexity** is linear in sequence length, so linear in pixel count. The full-attention baseline runs out of memory from 48×48 to 84×84; MambaIR scales like windowed attention with a global field.

Adapting a sequence model to a 2-D image

Mamba was built for text, which is a line. An image is a plane. The paper's contribution is the set of restoration-specific repairs needed to make the transfer work — and it names the two ways it fails without them.

The two failure modes^[1]

- **Local pixel forgetting.** To run the state-space equation, the feature map must be flattened into a sequence. Two pixels that are vertically adjacent may land hundreds of steps apart in that sequence, so the model forgets its own neighbourhood.
- **Channel redundancy.** The *excessive hidden state number* produces highly redundant channels, wasting capacity.
- **The fixes, inside the Residual State Space Block (RSSB):** a local convolution to restore neighbourhood awareness, and **channel attention** to suppress redundancy — plus a learnable factor controlling the skip connection.
- Ablation: removing the convolution costs Urban100 accuracy (34.15 → 34.04); removing convolution *and* channel attention gives 34.06; replacing the block with an MLP gives 34.22 against the 34.15 baseline configuration.

Scanning, structure, and results^[1]

- **Four-directional scan.** The feature map is unfolded in four directions and the results summed, so no single raster order dominates. Ablation: one-direction 34.06, two-direction 33.96, four-direction (baseline) 34.15 on Urban100.
- **Three stages**, structurally identical to SwinIR: shallow feature extraction (one 3×3 convolution) → deep feature extraction (stacked RSSBs grouped into RSSGs) → high-quality reconstruction aggregating shallow and deep features.
- **Colour denoising, Urban100** ($\sigma = 15/25/50$): SwinIR 35.13 / 32.90 / 29.82 · Restormer 35.13 / 32.96 / 30.02 · **MambaIR 35.37 / 33.21 / 30.30.**
- **Real noise:** SIDD 39.89 dB, DND **40.04 dB** — the best DND figure in the table, 0.01 dB above Restormer.
- **Super-resolution**, where the receptive field matters most: MambaIR outperforms SwinIR by **0.41 dB on Manga109 at ×2.**

A Stage 2 finding worth knowing. MambaIR's own repository changelog records that published complexity figures computed with `thop` were wrong and were later corrected. The model also cannot be imported at all without compiled CUDA extensions — `ModuleNotFoundError: mamba_ssm`. That is a deployment fact no paper reports, and it appears again on slide 78. ^[measured]

^[1] MambaIR24 abstract, §3.2, §4.2, Tables 1, 2, 5, 6 ^[2] repository `csguoh/MambaIR`, commit 33d7b34 ^[measured]

Three live answers, and no winner

The honest ending to Part V, and the reason this narrative closes on an open question rather than a conclusion. Each of these is current, each is credible, and they disagree about what the operator should be.

TRANSFORMER

Restormer[1] — move the attention axis

Attend over channels, keep depth-wise convolution for locality. Linear in pixels. **26.1 M parameters**. Best-in-class on Gaussian grayscale, first method past 40 dB on both SIDD and DND. Slowest of the three on CPU by a wide margin. [measured: 10.44 s at 256²][5]

STATE SPACE

MambaIR[3] — borrow from control theory

Global receptive field at linear cost via a selective SSM with four-directional scanning. Best DND result at 40.04 dB. Cannot run without compiled CUDA extensions, which is a genuine deployment constraint rather than a footnote. [measured: import fails][5]

CONVOLUTION

NAFNet[2] — delete the mechanisms

No attention, no activation functions. Claims the gains were never attention's to begin with. **116.0 M parameters and still 5.5× faster than Restormer on CPU**, because 1×1 and depth-wise convolutions map onto optimised GEMM paths. 40.30 dB on SIDD. [measured: 1.90 s at 256²][5]

DATA

SCUNet[4] — change the training distribution

The orthogonal answer, and arguably the most practical. **9.7 M parameters**, smallest of the four, with a synthetic degradation pipeline built to survive real photographs rather than benchmarks. [measured: 1.69 s at 256²][5]

Say this in the defence. A narrative that ends 'and then X won' would be false. The field currently holds three architectural answers and one data-side answer to the same bottleneck, published across three years, none of which has displaced the others. That is what an unresolved research question looks like from the inside.

[1] Restormer22 [2] NAFNet22 [3] MambaIR24 [4] SCUNet23 [5] parameter counts and CPU latencies [measured], see slide 77

VI

Engineering Reality

— PART VI

— In this part

-
- 01 The organising question
 - 02 Repositories, at pinned commits
 - 03 The prior, era by era, in code
 - 04 Frameworks, configuration, data pipelines
 - 05 Losses and pre-trained weights
 - 06 What we measured, and what broke
-

The same eighteen years, read again — this time from the source files rather than the papers.

Open the source file. Where is the prior?

Asked of one codebase per era, that single question produces a coherent engineering history instead of a set of disconnected repository summaries — and it answers, in passing, all four things the brief demands of a code deep-dive: structure, framework, data pipeline, and use of pre-trained weights.

ERA	OPEN THE FILE, AND YOU FIND...	WHAT SHIPS TO A USER	TEST-TIME COST
Classical DIP	A <code>while</code> loop and a hand-tuned <code>weight</code> argument	An algorithm	An optimisation, per image
Deep CNN, 2017	A list comprehension of convolution layers	A <code>.pth</code> file	One forward pass
Data branch, 2018–2025	The supervision — what goes in and what it is scored against. The network stops changing.	A <code>.pth</code> , plus ~50 gradient steps per image	A short optimisation, per image
Architecture branch, 2022–2024	A new operator replacing the 3×3 convolution	A <code>.pth</code> and a YAML config	One forward pass

The arc in one sentence. The prior moved out of the algorithm and into the weights — and then, on the data branch, partly back out of the weights and into a per-image optimisation that needs no clean data at all.

Repositories inspected, at pinned commits

Every code claim in this part was read from these trees at these commits. Pinning matters: `main` moves, and a claim about a moving branch is not reproducible.

MODEL	REPOSITORY	COMMIT	FRAMEWORK
Total variation (classical exemplar)	scikit-image 0.26.0, from PyPI	released version	NumPy
DnCNN	github.com/cszn/DnCNN	e93b278...	MatConvNet (MATLAB) + PyTorch port
DnCNN, SCUNet weights	github.com/cszn/KAIR	fc1732f...	PyTorch
Noise2Noise	github.com/NVlabs/noise2noise	7355519...	TensorFlow
Restormer	github.com/swz30/Restormer	68dc6ac...	PyTorch + BasicSR
NAFNet	github.com/megvii-research/NAFNet	2b4af71...	PyTorch + BasicSR
SCUNet	github.com/cszn/SCUNet	52e440a...	PyTorch
MambaIR	github.com/csguoh/MambaIR	33d7b34...	PyTorch + mamba_ssm CUDA ext.
P2N+	github.com/Li-Tong-621/P2N-plus	8a117db...	PyTorch

Two provenance notes that matter and are not in any paper. ZS-N2N has no GitHub repository at all — it was released solely as a Colab notebook. WNNM has only a third-party mirror, with no README and no licence. The availability of reference code is itself uneven across the field, and it is an obstacle to reproduction that citation counts do not show.

The classical prior is a while loop you can read

`scikit-image 0.26.0, restoration/_denoise.py, denoise_tv_chambolle`. The structure below is the Chambolle projection algorithm as implemented there.

skimage/restoration/_denoise.py — structure

```
def _denoise_tv_chambolle_nd(image, weight=0.1, eps=2.e-4, max_num_iter=200):
    2 p = np.zeros((image.ndim,) + image.shape, dtype=...) # dual variable
    3 g = np.zeros_like(p)
    4 i = 0
    5 while i < max_num_iter:          # ← the entire 'training'
    6     ...
    7     d = -p.sum(0)                # divergence of the dual field
    8     out = image + d
    9     E = (d ** 2).sum()
    10    ...
    11    norm = np.sqrt((g ** 2).sum(axis=0))[np.newaxis, ...]
    12    E += weight * norm.sum()      # ←  $\lambda \cdot \Phi(x)$ , the prior, one line
    13    tau = 1. / (2. * image.ndim)
    14    norm = norm * tau / weight
    15    norm += 1.
    16    p = p - tau * g
    17    p /= norm                    # projection onto the dual ball
    18    E /= float(image.size)
    19    if i == 0: E_init = E; E_previous = E
    20    else:
    21        if np.abs(E_previous - E) < eps * E_init: break # ← convergence
    22        else: E_previous = E
    23    i += 1
    24    return out
```

- 01 **Four things are visible on one screen, and they characterise the entire era.** (1) `weight` is λ — the prior's strength is a function argument a human passes. (2) `norm.sum()` is $\Phi(x) = \|\nabla x\|_1$ — the prior itself, in one line of arithmetic you can read and differentiate by hand. (3) The `while` loop is the test-time optimisation. (4) The `break` on relative energy change is the stopping criterion.
- 02 **Nothing persists.** There is no state, no file, no checkpoint. Call the function on a second image and every one of these iterations runs again from scratch. That is bottleneck 1, expressed as control flow.
- 03 **The contrast to hold in mind for the next slide.** Here the prior is *readable and untrainable*. In three slides it becomes *trainable and unreadable*. That trade is the whole of the deep era.

measured — `scikit-image 0.26.0`, installed from PyPI and read

DnCNN is a list comprehension

[cszn/KAIR, models/network_dncnn.py](#). The reference PyTorch implementation, by DnCNN's own first author. Verbatim.

KAIR/models/network_dncnn.py

```
class DnCNN(nn.Module):
    2 def __init__(self, in_nc=1, out_nc=1, nc=64, nb=17,
    3         act_mode='BR'):
    4     super(DnCNN, self).__init__()
    5     assert 'R' in act_mode or 'L' in act_mode
    6     bias = True
    7
    8     m_head = B.conv(in_nc, nc, mode='C'+act_mode[-1], bias=bias)
    9     m_body = [B.conv(nc, nc, mode='C'+act_mode, bias=bias)
    10              for _ in range(nb-2)]      # ← the whole network
    11     m_tail = B.conv(nc, out_nc, mode='C', bias=bias)
    12
    13     self.model = B.sequential(m_head, *m_body, m_tail)
    14
    15 def forward(self, x):
    16     n = self.model(x)
    17     return x-n                          # ← residual learning
```

- 01 **The architecture is one line.** `m_body` is a list comprehension of `nb-2` identical blocks. The three layer types from slide 32 are literally `m_head`, `m_body`, `m_tail`. There is no architectural cleverness to read because there is none in the paper.
- 02 **`mode='C'+act_mode` is the whole layer grammar.** `'CBR'` = Convolution → BatchNorm → ReLU for the body; `'CR'` = Convolution → ReLU for the head; `'C'` = Convolution alone for the tail. Exactly the paper's Fig. 1, encoded as a three-character string.
- 03 **`return x-n` is the paper's central idea, a single subtraction.** Note the subtlety: the *paper's network* outputs $\hat{v}$, and the subtraction happens outside. KAIR folds the subtraction inside `forward`, so the module returns the clean image. The residual mapping is still what the weights learn — only the boundary of the module moved.
- 04 **Where is the prior?** Nowhere in this file. It is in the `.pth` checkpoint this class will load. The file describes a *shape*; the prior is the tensor poured into it. This is the moment the prior stops being readable.

measured — [cszn/KAIR, models/network_dncnn.py](#), verbatim

Noise2Noise is an if-statement, and P2N is three forward passes

The strongest evidence that the data branch is a supervision story rather than an architecture story is that its code contains almost no architecture.

Noise2Noise, 2018 — NVlabs^[1]

- The repository is **TensorFlow** — the last major denoiser released outside PyTorch. Everything in Part V and everything after 2019 in Part IV is PyTorch. The migration is visible as a single dated boundary in the field's source history.
- The change from supervised training is structurally one branch: the target tensor is either the clean image or a second noisy image. The network, the optimiser, the schedule and the loss are untouched.
- **The theorem from slide 43 costs one conditional.** That asymmetry — a substantial statistical result, a trivial code change — is the whole reason the paper landed.

P2N+, 2025 — Li-Tong-621^[1]

- The network class is **UNet_n2n_un** and its channel widths match the 2018 Noise2Noise U-Net exactly. Seven years on, the backbone is named after its predecessor.
- `noise_estimation_loss` does not merely score an output — it **constructs the inputs**: run the model on y , estimate the noise, add it back with $+$ and $-$, then run the model twice more and compare the two *outputs*.
- **Three forward passes per step**, and passes two and three operate on tensors that did not exist before pass one. This is why calling P2N *a new loss function* is wrong.
- One function implements **all** the competing self-supervised paradigms behind a `--mode` flag. The whole data branch fits in one file.

A caveat we found in the code and would not overstate. Two of those `--mode` branches feed RDC-constructed data to Noise2Void and Noise2Noise. The paper itself states this data is unsuitable for them, because the noise in RDC-constructed images is **correlated**, which collapses their training. Those branches are a documented negative result, not a fair comparison — and reading the flag table without reading the paper would lead you to the opposite conclusion.

^[1] measured — `NVlabs/noise2noise` commit 7355519; `Li-Tong-621/P2N-plus` commit 8a117db ^[2] P2N25 on RDC data unsuitability

One **rearrange** is the difference between $O(HW \cdot HW)$ and $O(C \cdot C)$

swz30/Restormer, basicsr/models/archs/restormer_arch.py, Attention.forward.

Verbatim. The claim of linear complexity is not in a comment — it is in the tensor layout.

Restormer/basicsr/models/archs/restormer_arch.py

```
def forward(self, x):
    2  b, c, h, w = x.shape
    3
    4  qkv = self.qkv_dwconv(self.qkv(x)) # 1x1 conv, then 3x3 depth-wise
    5  q, k, v = qkv.chunk(3, dim=1)
    6
    7  q = rearrange(q, 'b (head c) h w -> b head c (h w)', head=self.num_heads)
    8  k = rearrange(k, 'b (head c) h w -> b head c (h w)', head=self.num_heads)
    9  v = rearrange(v, 'b (head c) h w -> b head c (h w)', head=self.num_heads)
    10
    11 q = torch.nn.functional.normalize(q, dim=-1)
    12 k = torch.nn.functional.normalize(k, dim=-1)
    13
    14 attn = (q @ k.transpose(-2, -1)) * self.temperature # ← (C/head) x (C/head)
    15 attn = attn.softmax(dim=-1)
    16
    17 out = (attn @ v)
    18 out = rearrange(out, 'b head c (h w) -> b (head c) h w',
    19                 head=self.num_heads, h=h, w=w)
    20 return self.project_out(out)
```

- 01 **Read the einops pattern.** 'b (head c) h w -> b head c (h w)' puts **pixels on the last axis and channels second-to-last**. In standard spatial attention it is the other way round.
- 02 **Therefore $q @ k.transpose(-2, -1)$ has shape $(C/head) \times (C/head)$,** not $(HW) \times (HW)$. At level 1 with $dim = 48$ and 1 head, that is a 48×48 matrix regardless of whether the image is 256×256 or 4000×3000 . The paper's linear-complexity claim is exactly this line.
- 03 **F.normalize along the pixel axis** makes the product a cosine similarity between channels — the cross-covariance the paper describes — and `self.temperature` is a learned per-head scale replacing the fixed $1/V_d k$.
- 04 **qkv_dwconv is the locality the paper insists on**, and the ablation on slide 55 measures its worth at 0.15 dB. Removing one `groups=dim * 3` convolution costs more than most architectural papers gain.
- 05 **Why it is nonetheless slow on CPU.** Three `rearrange` calls force tensor-layout changes, and attention matmuls are poorly served by a CPU backend. Measured: **10.44 s at 256×256** , the slowest model we ran. [\[measured\]](#)

measured — swz30/Restormer, restormer_arch.py, Attention.forward, verbatim

SimpleGate and SCA are smaller than their descriptions

[megvii-research/NAFNet, basicsr/models/archs/NAFNet_arch.py](https://github.com/megvii-research/NAFNet/blob/main/basicsr/models/archs/NAFNet_arch.py). Verbatim. The paper spends two pages deriving these; the implementation is four lines and a [Sequential](#).

NAFNet/basicsr/models/archs/NAFNet_arch.py

```
class SimpleGate(nn.Module):
    2 def forward(self, x):
    3     x1, x2 = x.chunk(2, dim=1)
    4     return x1 * x2                # ← replaces GELU entirely
    5
    6 Simplified Channel Attention, inside NAFBlock.__init__:
    self.sca = nn.Sequential(
    8     nn.AdaptiveAvgPool2d(1),
    9     nn.Conv2d(dw_channel // 2, dw_channel // 2, kernel_size=1, bias=True),
    )0                                # ← no ReLU, no Sigmoid
    11
    12 NAFBlock.forward, first half:
    x3= self.norm1(inp)
    x4= self.conv1(x)                # 1x1, expand
    x5= self.conv2(x)                # 3x3 depth-wise (groups=dw_channel)
    x6= self.sg(x)                   # SimpleGate: halves the channels
    x7= x * self.sca(x)              # SCA
    x8= self.conv3(x)                # 1x1, project back
    y0= inp + x * self.beta          # learned residual scale
```

- 01 **x.chunk(2, dim=1) then x1 * x2.** Compare with GELU's implementation: $0.5x(1+\tanh[\sqrt{2/\pi}(x+0.044715x^3)])$. The paper's claim that the activation is unnecessary is not rhetoric — the replacement is genuinely a channel split and a multiply.
- 02 **self.sca contains no nonlinearity at all.** Compare to standard channel attention, which is pool $\rightarrow 1\times 1 \rightarrow \text{ReLU} \rightarrow 1\times 1 \rightarrow \text{Sigmoid}$. Two activations removed, and slide 58 measures the result as **+0.03 dB**, not a loss.
- 03 **self.beta and self.gamma are learned scalars initialised to zero — skip-init**, which means the block starts as an identity and training decides how much of it to switch on. This is a stability device, not an architectural claim, and it does not appear in the paper's block diagram.
- 04 **Where the parameters live.** Width doubles per level, so the 12 middle blocks run at 1024 channels; each **NAFBlock** costs roughly $7c^2$, giving $12 \times 7 \times 1024^2 \approx 88 \text{ M}$ of the 116 M total in the bottleneck alone. [\[measured\]](#)
- 05 **Why it is fast despite being large.** 1×1 and depth-wise convolutions map onto highly optimised GEMM kernels. Measured: **1.90 s at 256×256 — 5.5× faster than Restormer with 4.4× the parameters.** [\[measured\]](#)

measured — megvii-research/NAFNet, NAFNet_arch.py, verbatim

MambaIR's block — and why SCUNet has no code slide

The code deep-dive reads one representative file per bottleneck, so the two architecture-branch papers that are not DnCNN, Restormer or NAFNet are read where their contribution lives. SCUNet's is its degradation pipeline — **slide 73** — not its Swin-conv U-Net. MambaIR's is a state-space operator that lives in a compiled CUDA dependency; the operator itself is a kernel call, but the block that wraps it into an image model reads plainly. Here it is.

read — [csguoh/MambaIR](#), [basicsr/archs/mambair_arch.py](#), at the commit on slide 65

MambaIR's block — and why SCUNet has no code slide

MambaIR/basicsr/archs/mambair_arch.py — VSSBlock + CAB (condensed: reshape/permute plumbing and DropPath elided; the two-branch residual is faithful)

```
class VSSBlock(nn.Module):          # the paper's RSSB
2   def __init__(self, dim, d_state=16):
3       self.ln_1 = LayerNorm(dim)
4       self.self_attention = SS2D(dim, d_state) # scan
5       self.skip_scale = Parameter(ones(dim)) # factor
6       self.conv_blk = CAB(dim) # conv + CA
7       self.ln_2 = LayerNorm(dim)
8       self.skip_scale2 = Parameter(ones(dim))
9
10  def forward(self, x, x_size):
11      x = x.view(B, *x_size, C) # [B,H,W,C]
12      h = self.ln_1(x)
13      x = x*self.skip_scale + self.self_attention(h)
14      x = x*self.skip_scale2 + self.conv_blk(self.ln_2(x))
15      return x.view(B, -1, C)
16
#7CAB - the local branch, no state-space at all:
CAB = nn.Sequential(
19  nn.Conv2d(dim, dim//r, 3,1,1), nn.GELU(),
20  nn.Conv2d(dim//r, dim, 3,1,1),
21  ChannelAttention(dim)) # pool->1x1->ReLU->1x1->Sig
```

- 01 **Two branches, one block.** MambaIR's RSSB (class `VSSBlock`) is not a Mamba layer dropped into a U-Net. It is a state-space branch (`SS2D`) for global context *and* a convolution-plus-channel-attention branch (`CAB`) for the local detail the scan discards, each wrapped in a learnable-scaled residual.
- 02 **SS2D is the operator — and the reason it would not run.** `SS2D` is the four-directional selective scan of slide 61; it is the one call into the compiled `mamba_ssm` CUDA extension, which is exactly why the model could not be instantiated in our benchmark (slide 77). The scan is the mechanism; the block above is only how it is wired into an image model.
- 03 **conv_blk = CAB is ordinary.** Two 3×3 convolutions, a `GELU`, and an RCAN-style channel attention (pool → 1×1 → ReLU → 1×1 → Sigmoid). This is the local convolution and channel attention whose ablation on slide 61 is worth **+0.11 dB** on Urban100.
- 04 **skip_scale / skip_scale2 are the learnable factors** named on slide 61 — one per branch, initialised to ones, so the block starts near identity and training decides how much of each branch to admit.

read — [csguoh/MambaIR](#), [basicsr/archs/mambair_arch.py](#), at the commit on slide 65

MambaIR's block — and why SCUNet has no code slide

- 05 **Why SCUNet is read on slide 73, not here.** SCUNet's contribution is its degradation pipeline, not its Swin-conv U-Net — so its code that matters is the dataloader, read where the data pipelines are. Showing `SCUNet.forward` would show the least novel part of the paper. The code section reads one representative file per bottleneck, and these two papers are read where their contribution actually lives.

read — [csguoh/MambaIR](#), [basicsr/archs/mambair_arch.py](#), at the commit on slide 65

How the field's software stack moved, dated by repository

The brief asks what frameworks these models use and how pre-trained weights are used today compared with older manual methods. The answer is best given as a migration with dates attached to actual commits.

ERA	FRAMEWORK	CONFIGURATION	HOW A MODEL IS INSTANTIATED
Classical, ≤ 2014	MATLAB / C, or NumPy	Function arguments	Call a function with tuned constants
DnCNN, 2017	MatConvNet (MATLAB)	Hard-coded in a training script	Edit the script, rerun
Noise2Noise, 2018	TensorFlow	<code>argparse</code> flags	Command line
KAIR, 2018–2024	PyTorch	A JSON options file	Name a model string, options resolve it
Restormer / NAFNet, 2022	PyTorch + BasicSR	YAML , resolved by a registry	<code>build_network(opt)</code> reads <code>type:</code> from YAML
SCUNet, 2023	PyTorch	Constructor arguments	Direct instantiation
MambaIR, 2024	PyTorch + compiled CUDA ext.	YAML + BasicSR registry	Requires <code>mamba_ssm</code> to import at all
P2N+, 2025	PyTorch	<code>argparse</code> , with a <code>--mode</code> flag	One flag selects the training paradigm

Two observations worth making aloud. (1) The registry pattern in BasicSR means the *architecture is named by a string in a config file*, so swapping Restormer for NAFNet is a one-line YAML edit — architectures have become interchangeable components. (2) Between 2017 and 2022 the unit of reproducibility moved from *a script you edit* to *a config you version-control*, which is why modern papers can honestly claim exact reproduction.

measured — repositories at the commits on slide 65

Where the noise is added, and why that choice matters

The brief asks how modern data pipelines work. The interesting axis is not the [Dataset](#) boilerplate — it is *when* the degradation is applied, because that determines what the network can possibly learn.

The two strategies

- **Offline.** Generate noisy/clean pairs once, write them to disk, and read them during training. Deterministic and fast, but the network sees each noise realisation many times and can memorise it.
- **On-the-fly, in `__getitem__`.** Draw a fresh noise realisation for every sample, every epoch. The network never sees the same corrupted patch twice, which is a powerful implicit regulariser — and it is free.
- **DnCNN, 2017:** patches cropped from 400 images, noise added per sample, σ drawn from $[0, 55]$ for the blind model. $128 \times 3\,000$ patches of 50×50 .^[1]
- **SCUNet, 2023:** the entire degradation chain — Gaussian, Poisson, speckle, JPEG, ISP-simulated sensor noise, resizing, with random shuffling and double degradation — is executed **per sample, at load time**. The dataloader is the contribution.^[2]

What is in every modern pipeline

- **Random crop** to a fixed patch size. 40–55 px in DnCNN; 128 px rising to 384 px in Restormer's^[3] progressive schedule; 256 px in NAFNet^[5]; 128 px in MambaIR's^[4] denoising configuration.
- **Geometric augmentation only.** Horizontal and vertical flips, and $90^\circ/180^\circ/270^\circ$ rotations. Notably absent: colour jitter, cutout, mixup — anything that alters intensity statistics would corrupt the very quantity being estimated.
- **Batch size traded against patch size.** Restormer explicitly reduces batch size as patch size grows *to maintain a similar time per optimisation step*.
- **Real-noise datasets skip synthesis entirely.** SIDD ships 320 captured noisy/clean scene pairs. There is no noise model in that pipeline — the camera is the noise model.

The one line to take away. In 2017 the data pipeline was infrastructure. By 2023 — in SCUNet, and in the whole data branch of Part IV — the data pipeline *is the method*.

^[1] DnCNN17 §IV-A ^[2] SCUNet23 §3.2 ^[3] Restormer22 §4 ^[4] MambaIR24 §4.1 ^[5] NAFNet22 §5.1

What modern denoisers actually optimise — and what they conspicuously do not

The brief asks specifically about the modern loss functions being used. The finding is a negative one and it is more interesting than the positive list.

RESTORMER

Plain L1

L1 loss for 300K iterations.[1] No perceptual term, no adversarial term. The most-cited restoration Transformer of the decade optimises mean absolute error.

P2N

Annealed ℓ_r

An ℓ_r loss with r **decreasing during training** from 2 toward 1.5 or 0 — moving the estimator from the mean toward the median and beyond, as the model's own estimates become trustworthy. [measured][3]

NAFNET

PSNR loss

The negative of the evaluation metric, computed per image before the batch mean. Optimising the metric directly — honest, and a reminder that the metric is the target.[2]

ABSENT

Perceptual and adversarial losses

Essentially absent from denoising, in contrast to super-resolution, where SRGAN-style losses are standard. **Reason:** denoising is evaluated by fidelity to a known ground truth, and a GAN that hallucinates plausible texture scores worse even when it looks better. The metric constrains the loss.[4]

The honest caveat. That last card is an inference from what we read across five repositories, not a claim any of these papers makes. It is labelled as a reading, and if a TA challenges it, the correct response is that we observed the absence rather than that a paper asserted it.

[1] Restormer22 §4 [2] NAFNet22 §5.1 [3] measured — P2N-plus loss code [4] reading

The prior became a file you download

This is the brief's sharpest comparison: how pre-trained weights are used today versus older manual feature-extraction methods. The contrast is total, and it is best stated in terms of what each era ships.

Then — the classical artefact^[2]

- **What ships:** an algorithm, plus constants a human chose.
- **Reuse:** none. Every image restarts from nothing. Nothing computed on image 1 helps image 2.
- **Adaptation to a new domain:** re-derive the prior, or re-tune the constants by hand, and hope.
- **Reproducibility:** depends on matching the author's parameter table, which is often in prose rather than code.
- **Where the knowledge lives:** in the paper.

Now — the modern artefact^[1]

- **What ships:** a `.pth` file — a serialised `state_dict`, a dictionary mapping layer names to tensors. Downloaded once, reused indefinitely.
- **Loading:** `model.load_state_dict(torch.load(path))`, with `strict=True` by default so a shape mismatch fails loudly rather than silently.
- **Distribution:** GitHub Releases, Google Drive links, and increasingly Hugging Face model cards that carry the architecture, the training data, the metrics and the licence as structured metadata.
- **Adaptation:** fine-tune, or use the weights as an initialisation — which is precisely what P2N does before its per-image optimisation.
- **Where the knowledge lives:** in the tensors. The repository describes only a shape.

The consequence that is easy to miss. Because the weights are a file and the architecture is a string in a YAML, a modern denoiser is *two decoupled artefacts*. You can ship a better checkpoint without touching the code, and swap the architecture without retraining the pipeline. The classical era had no such separation, and that is a large part of why it could not compound.

^[1] measured — checkpoint loading in KAIR, Restormer, NAFNet, SCUNet ^[2] brief §2 on pre-trained weights

Every model pads — and they all pad differently

A U-Net that downsamples k times requires input dimensions divisible by 2^k . Every architecture-branch model solves this with the same pad-then-crop idiom and a different constant, which becomes a real problem the moment two are chained in one pipeline.

MODEL	REQUIRED MULTIPLE	PADDING MODE	WHERE
Restormer	8	<code>reflect</code>	<code>demo.py</code>
NAFNet	16 — computed as <code>2 ** len(self.encoders)</code>	zero (default <code>F.pad</code>)	<code>NAFNet_arch.check_image_size</code>
P2N+	32	<code>reflect</code>	<code>test.py</code>
SCUNet	64 — <code>int(np.ceil(h/64) * 64-h)</code>	<code>nn.ReplicationPad2d</code>	<code>network_scunet.forward</code>

A factor of eight separates the extremes, and the padding mode differs too — zero padding injects a hard artificial edge, reflection and replication do not. Feed a 1000×1000 image to SCUNet and 26 rows and columns of replicated content are silently invented, processed, and cropped away. None of this appears in any of the four papers. It is exactly the class of fact the brief means by *practical deployment realities*, and it is only visible by reading `forward`.

measured — verbatim from the four repositories at the commits on slide 65

Seven architectures, instantiated and timed

Each model was instantiated from its own repository at the commit on slide 65, in the configuration that repository uses for denoising. Parameters are `sum(p.numel() for p in model.parameters())`. Latency is the mean of three forward passes at 256×256 after one warm-up, under `torch.no_grad()`, **CPU only**, PyTorch 2.13.0, no CUDA.

MODEL	CONFIGURATION	PARAMETERS	CPU LATENCY, 256 ²
DnCNN-B	<code>DnCNN(in_nc=1, nc=64, nb=20)</code>	0.668 M	1.015 s
P2N+ backbone, SPI off	<code>UNet_n2n_un(conv_type='plain')</code>	0.991 M	0.286 s
P2N+ backbone, SPI on	<code>UNet_n2n_un(conv_type='SPI')</code>	0.925 M	0.710 s
SCUNet	<code>SCUNet(in_nc=3, config=[2] * 7, dim=64)</code>	9.663 M	1.685 s
Restormer	paper defaults, dim=48, [4,6,6,8]	26.127 M	10.436 s
NAFNet	SIDD width=64, enc [2,2,4,8], mid 12	115.983 M	1.899 s
MambaIR	—	<i>not measurable</i>	import fails

Two caveats, stated before they are asked. (1) Absolute CPU latencies are **not** comparable to published GPU timings — the *ratios* are the informative quantity. (2) The forward passes used **random tensors**, so no PSNR or SSIM was measured here; every quality number in this deck comes from the papers, and every timing and parameter count on this slide comes from us. The two are never mixed in one column.

Four conclusions, none of them visible from a paper

These are the readings that justify having run the code at all. Each contradicts something a reader would reasonably assume from the published tables.

A

Parameters do not predict latency

NAFNet carries **4.4× Restormer's parameters and runs 5.5× faster** on identical hardware. NAFNet is 1×1 and depth-wise convolutions on fast GEMM paths; Restormer's `rearrange` calls force layout changes and its attention matmuls are poorly served by a CPU backend. **'More efficient' is meaningless without naming the metric — parameters, MACs, or wall-clock — and the device.**^[1]

C

The oldest model is the smallest and still not the fastest

DnCNN is the smallest model in the table at 0.668 M and is nonetheless **slower than the 0.991 M P2N+ backbone**, because DnCNN is 20 sequential full-resolution 3×3 convolutions at 64 channels while a U-Net does most of its work at reduced resolution.^[1] Depth at full resolution is expensive in a way parameter count does not show.

B

SPI costs latency, not parameters

Switching Symmetric Prior Injection on **reduces** P2N+'s parameters by 6.7% (0.991 M → 0.925 M), because `SymLayer` routes through a half-width bottleneck — yet latency rises **2.5×** (0.286 s → 0.710 s), because every layer is evaluated twice, on x and on -x.^[1] This is the exact opposite of what a parameter table predicts, and it is invisible unless you run it.

D

Some models simply do not run

MambaIR raises `ModuleNotFoundError: mamba_ssm` without compiled CUDA extensions.^[1] That is not a gap in our experiment — it *is* the result. A model that cannot be imported on a CPU machine has a deployment profile fundamentally different from one that can, and no paper reports this. The repository's own changelog^[2] also records that published complexity figures computed with `thop` were wrong.

^[1] measured — Stage 2 benchmark ^[2] MambaIR repository changelog

VII

Comparative Analysis

— PART VII

— In this part

- 01 The rules we followed
- 02 BSD68 — the long ladder
- 03 Urban100 — where the gap is
- 04 Real noise — where the assumption breaks
- 05 Efficiency
- 06 Supervision cost
- 07 The whole arc in one view

Eighteen years of numbers, assembled under rules strict enough that the comparison means something.

Five rules, so that the comparison survives scrutiny

Cross-paper PSNR tables are the most commonly abused artefact in this field. These are the constraints we imposed on ourselves, and they are worth stating before any number appears.

-
- 01 **1 • No number was computed by us.** Every PSNR and SSIM in Part VII is reproduced from a published table, with the paper and table named. We ran no denoising evaluation of our own, because doing it correctly would require matching each paper's crop, border, colour-space and rounding conventions — and a table that mixes our numbers with theirs would be worse than useless.
 - 02 **2 • Rows come from one table wherever possible.** When comparing DnCNN to Restormer, both figures are taken from Restormer's Table 4, where the authors evaluated both under one protocol. Taking DnCNN's number from its own paper and Restormer's from its own would silently compare two different evaluation scripts.
 - 03 **3 • Timings and quality are never mixed in one column.** Every quality number is from a paper; every latency and parameter count is from our own measurements on stated hardware. The two are reported in separate tables and labelled.
 - 04 **4 • Device asymmetries are stated.** The DnCNN runtime table compares CPU figures for classical methods against GPU figures for learned ones, and the paper itself flags that GPU times were copied from other papers. We reproduce the caveat every time we reproduce the numbers.
 - 05 **5 • Where a figure could not be verified, it is absent.** We found one probable extraction error in a published table we could not confirm (MambaIR's CBSD68 $\sigma=25$ cell, which breaks the pattern of its own row). That cell is excluded from every chart in this deck rather than reproduced with a footnote.

BSD68 grayscale — eighteen years, one benchmark

Average PSNR in dB. The 2007–2017 block is from DnCNN Table II; the 2017–2022 block is from Restormer Table 4, which re-evaluated DnCNN under its own protocol. The two DnCNN rows differ slightly for exactly that reason, and that difference is itself worth knowing.

METHOD	YEAR	ERA	$\Sigma = 15$	$\Sigma = 25$	$\Sigma = 50$	SOURCE
BM3D	2007	Classical	31.07	28.57	25.62	DnCNN T2[1]
EPLL	2011	Classical	31.21	28.68	25.67	DnCNN T2[1]
MLP	2012	Early learning	—	28.96	26.03	DnCNN T2[1]
WNNM	2014	Classical peak	31.37	28.83	25.87	DnCNN T2[1]
TNRD	2016	Bridge	31.42	28.92	25.97	DnCNN T2[1]
DnCNN	2017	Deep CNN	31.73	29.23	26.23	DnCNN T2[1]
DnCNN (re-evaluated)	2017	Deep CNN	31.62	29.16	26.23	Restormer T4[2]
FFDNet	2018	Deep CNN	31.63	29.19	26.29	Restormer T4[2]
DRUNet	2021	Deep CNN	31.91	29.48	26.59	Restormer T4[2]
SwinIR	2021	Transformer	31.97	29.50	26.58	Restormer T4[2]
Restormer	2022	Transformer	31.96	29.52	26.62	Restormer T4[2]

Read the $\sigma = 25$ column as a single story. 2007 → 2014, seven years of hand-designed priors: **+0.26 dB**. 2014 → 2017, one architectural idea: **+0.40 dB**. 2017 → 2022, five years and an entire Transformer revolution: **+0.36 dB**. On ordinary natural images the field is deep into diminishing returns, and has been since roughly 2018. **This table is the argument that BSD68 has stopped being informative** — which is why the next two slides exist.

[1] DnCNN17 Table II [2] Restormer22 Table 4 — source column names the origin of every row

Urban100 — the same five years, five times the gain

Identical models, identical table, identical noise level. The only thing that changes is the dataset
— from ordinary natural images to photographs of buildings, where structure repeats across
hundreds of pixels.

+0.39 dB

BSD68, $\Sigma = 50 \cdot \text{DNCNN} \rightarrow \text{RESTORMER}$

26.23 $\rightarrow$ 26.62. Ordinary natural images, little long-range repetition. Five years of architectural work buys almost nothing.^[1]

+1.98 dB

URBAN100, $\Sigma = 50 \cdot \text{DNCNN} \rightarrow \text{RESTORMER}$

26.35 $\rightarrow$ 28.33. Highly repetitive urban structure. The same five years buy **five times as much**.^[1]

−0.71 dB

BARBARA, $\Sigma = 25 \cdot \text{DNCNN VS BM3D, 2017}$

30.00 vs 30.71. The identical phenomenon, visible in the anchor paper's own table five years earlier — on the one image in Set12 whose texture repeats.^[2]

- 01 **These three numbers are the same fact, measured three times across eight years.** A convolutional network with a 35×35 receptive field cannot exploit self-similarity beyond 35 pixels. Non-local classical methods could. Attention gave it back.
- 02 **The corollary, which is the more useful claim.** The modern architecture branch did not make denoising uniformly better. It made denoising better **specifically where non-locality matters**. Aggregate benchmark numbers conceal this completely, which is why a per-dataset reading is not pedantry — it is the only way to see what actually changed.
- 03 **And it explains why BSD68 saturated.** BSD68 contains relatively little long-range repetition, so it measures a capability the field stopped competing on around 2018 while being blind to the one it moved to.

^[1] Restormer22 Table 4 (BSD68 and Urban100 columns) ^[2] DnCNN17 Table III (Barbara) — ratio computed by us from published values

Real sensor noise — and a 16 dB collapse

SIDD and DND contain photographs of real scenes taken with real smartphone cameras, where noise is signal-dependent, spatially correlated, and processed by an imaging pipeline. Both tables below are from Restormer's Table 6, evaluated under one protocol.

METHOD	TYPE	SIDD PSNR	SIDD SSIM	DND PSNR	DND SSIM
DnCNN	AWGN-trained CNN, 2017	23.66	0.583	32.43	0.790
BM3D	Classical, 2007	25.65	0.685	34.51	0.851
CBDNet*	Real-noise CNN, 2019	30.78	0.801	38.06	0.942
MPRNet	Multi-stage CNN, 2021	39.71	0.958	39.80	0.954
Uformer	Transformer, 2022	39.77	0.959	39.96	0.956
Restormer[1]	Transformer, 2022	40.02	0.960	40.03	0.956
NAFNet[2]	Convolutional, 2022	40.30	—	—	—
MambaIR[3]	State-space, 2024	39.89	0.960	40.04	0.956

The single most important number in this deck: **23.66 dB**. DnCNN — which beats BM3D by 0.6 dB on synthetic Gaussian noise — is beaten by BM3D by **1.99 dB** on real sensor noise, and trails Restormer by **16.36 dB**. A model trained on the wrong noise distribution does not degrade gracefully; it collapses. This is exactly what SCUNet means by *noise assumption mismatch*, and it is why the field's centre of gravity moved from AWGN benchmarks to real-noise benchmarks between 2019 and 2022. *NAFNet's 40.30 dB is from its own abstract, evaluated on SIDD under the same benchmark but not in Restormer's table; the asterisk marks methods using additional training data.

[1] Restormer22 Table 6 [2] NAFNet22 abstract [3] MambaIR24 Table 6 [4] SCUNet23 §1

Cost, measured three different ways — with three different answers

Quality is only half of a comparison. The left block is reproduced from the DnCNN paper; the right block is our own measurement. They are kept separate deliberately, per rule 3.

METHOD	ERA	PUBLISHED 512 ² RUNTIME ^[1]	DEVICE	PARAMETERS [MEASURED] ^[2]	CPU 256 ² [MEASURED] ^[2]
WNNM	Classical peak	773.2 s	CPU	— (no network)	—
EPLL	Classical	45.5 s	CPU	— (no network)	—
BM3D	Classical	2.85 s	CPU	— (no network)	—
TNRD	Bridge	1.33 s / 0.032 s	CPU / GPU	— (not run)	—
DnCNN-B	Deep CNN	4.11 s / 0.060 s	CPU / GPU	0.668 M	1.015 s
P2N+ backbone	Data branch	— (not published)	—	0.991 M	0.286 s
SCUNet	Architecture	— (not published)	—	9.663 M	1.685 s
Restormer	Architecture	— (not published)	—	26.127 M	10.436 s
NAFNet	Architecture	— (not published)	—	115.983 M	1.899 s

Three incompatible rankings from one set of models. By **parameters**, DnCNN is best and NAFNet is worst by 174×. By **CPU wall-clock**, P2N+'s backbone is best and Restormer is worst by 36×. By **quality on real noise**, NAFNet is best and DnCNN is worst by 16 dB. There is no single efficiency axis, and a paper that reports only one has chosen the one that flatters it. Restormer's own claim — **3.14× fewer FLOPs and 13× faster than SwinIR** — is a fourth axis again.

^[1] DnCNN17 Table IV ^[2] measured, Stage 2 ^[3] Restormer22 §4.4

What supervision costs, in decibels

Parts IV and V are usually compared on quality alone, which misses the point of Part IV entirely. The right comparison sets quality against *what the method demands of you before it will run at all*.

METHOD	WHAT IT REQUIRES	BSD68 $\Sigma=25$	SOURCE
Supervised U-Net (traditional)	Paired clean + noisy images	29.06	N2V19 Fig. 4[1]
DnCNN-S	Paired clean + noisy images	29.23	DnCNN17 T2[2]
Noise2Noise	Two independent noisy copies per scene	28.86	N2V19 Fig. 4[1]
BM3D	Nothing — no training at all	28.57 / 28.59	DnCNN17 T2[2] / N2V19[1]
Noise2Void	A body of single noisy images	27.71	N2V19 Fig. 4[1]

Read the whole ladder in one sentence. Giving up the clean target costs about **0.20 dB** (29.06 → 28.86). Giving up the second noisy copy costs a further **1.15 dB** (28.86 → 27.71) and drops below a 2007 algorithm. **But on cryo-TEM and the Cell Tracking Challenge datasets, the three rows above Noise2Void simply do not exist** — the paper's own figure captions the ground-truth column *does not exist*. Where clean data is available, supervision wins. Where it is not, the comparison is not between 27.71 and 32.23; it is between 27.71 and nothing.

[1] N2V19 §4.1 and Fig. 4 [2] DnCNN17 Table II — all values as printed

Eighteen years, seven methods, one table

The comparative analysis compressed to the rows that carry the argument. Read down the last two columns: the story is not that the numbers went up, but that *what they are measuring* changed.

METHOD	YEAR	WHERE THE PRIOR LIVES	BSD68 $\Sigma=50$	URBAN100 $\Sigma=50$	SIDD (REAL)
BM3D ^[1]	2007	Hand-written: NSS + 3-D sparsity	25.62	—	25.65
WNNM ^[1]	2014	Hand-written: NSS + low rank	25.87	—	—
TNRD ^[1]	2016	Learned, inside a fixed PDE	25.97	—	—
DnCNN ^[1]	2017	Learned, in 0.67 M weights	26.23	26.35	23.66
DRUNet ^[2]	2021	Learned, multi-scale CNN	26.59	27.96	—
Restormer ^[2]	2022	Learned, channel attention	26.62	28.33	40.02
NAFNet ^[3]	2022	Learned, no attention at all	—	—	40.30
MambaIR ^[4]	2024	Learned, state-space scan	—	—	39.89

Three readings, and they are the conclusion of Part VII. (1) On BSD68 the field gained **1.00 dB in fifteen years** and has plainly saturated. (2) On Urban100 it gained **1.98 dB in five years**, because that benchmark measures the capability the field actually moved to. (3) On real noise it gained **16.64 dB**, because the first four rows were answering a question — remove AWGN — that real photographs never asked. **Progress in denoising since 2017 is mostly not progress on the 2017 problem.**

^[1] DnCNN17 Table II ^[2] Restormer22 Tables 4 and 7 ^[3] NAFNet22 abstract ^[4] MambaIR24 Table 6

VIII

Open Problems & Conclusion

— PART VIII

— In this part

- 01 Real noise modelling
- 02 The perception–distortion trade-off
- 03 Efficiency and deployment
- 04 Evaluation itself
- 05 Self-supervision's remaining gap
- 06 What we would build today
- 07 The thesis, restated

What eighteen years did not solve, and what we would build tomorrow morning.

Real noise, and the perception–distortion trade-off

The two open problems for which this deck already contains hard evidence, rather than speculation.

OPEN PROBLEM 1

A general-purpose blind real denoiser

SCUNet's^[1] abstract states the position as of 2023: *a general-purpose blind denoising method for real images remains unsolved*. The evidence on slide 83 is stark — DnCNN scores 23.66 dB on SIDD. SCUNet's answer is to synthesise realistic degradations; Restormer and NAFNet's answer is to train directly on SIDD's 320 captured scenes.^[3] **Neither generalises to an unseen camera with an unseen ISP**, and no benchmark currently measures that.

OPEN PROBLEM 2

Fidelity metrics reward blur

Every loss in Part VI is L1, L2 or PSNR. All three recover a conditional mean, and the conditional mean of the plausible clean images is smoother than any of them. **Noise2Noise^[2] names this mechanism explicitly**: an L2-trained regressor *learns to output the average of all plausible explanations, which results in spatial blurriness*. The field's default objective is structurally biased toward the very over-smoothing that BM3D was criticised for.

WHY IT IS HARD

Noise is a pipeline, not a distribution

Real noise is read noise plus shot noise in the raw domain, then transformed by demosaicing, white balance, colour conversion, tone mapping and gamma — each of which correlates it spatially and across channels. SCUNet^[1] models this by *inverting* the ISP, adding noise in raw, and running it forward again. That is an engineering simulation of a physical device, not a probability model, and it is as good as the device model it assumes.

WHY IT PERSISTS

The metric constrains the loss

Perceptual and adversarial losses would fix the blur and would score *worse* on PSNR, because they hallucinate plausible texture that does not match the ground truth pixel for pixel. As long as leaderboards are PSNR-ranked, the incentive points the wrong way. This is why, as slide 74 observes, perceptual losses are common in super-resolution and essentially absent from denoising.^[4]

^[1] SCUNet23 abstract, §3.2 ^[2] N2N18 §2 ^[3] Restormer22 Table 6 ^[4] reading

Efficiency, evaluation, and the self-supervision gap

Three further open problems, each of which this deck has produced direct evidence for rather than inherited from a survey.

OPEN PROBLEM 3

Nobody agrees what efficient means

Slide 76 produced three incompatible rankings of the same models from parameters, wall-clock and quality — and Restormer's[4] FLOP claim is a fourth. MambaIR's own changelog concedes its published [thop](#) complexity figures were wrong. **The field has no agreed cost metric**, which makes efficiency claims across papers close to meaningless and is why our benchmark reports the counting method alongside every number.[1]

OPEN PROBLEM 5

Self-supervision has not met supervision

N2V[2] trails supervised training by 1.35 dB on BSD68 and sits below BM3D. P2N's[3] contribution is removing the *information loss*, not closing the quality gap, and it still pays per-image test-time optimisation. **Bottleneck 2 is mitigated, not closed** — and the domains that need it most, microscopy and medical imaging, are precisely those where no clean reference exists to measure the remaining gap.

OPEN PROBLEM 4

The benchmarks have saturated

BSD68 $\sigma=25$ moved 0.36 dB in the five years from DnCNN to Restormer[4], while Urban100 moved five times as much and SIDD moved sixteen decibels. A benchmark that cannot distinguish 2017 from 2022 is no longer measuring the thing the field is working on. **New capability is arriving faster than the instruments to detect it.**

OPEN PROBLEM 6

The two branches have never merged

This is our own observation and the structural claim of the deck. P2N[3] ships the 2018 Noise2Noise U-Net verbatim; Restormer, NAFNet and MambaIR[4] all assume paired data. **Nobody has published a state-space or Transformer backbone trained by RDC/DCS supervision.** Whether the branches compose or conflict is, as far as we found, an open and apparently untested question.

[1] measured, Stage 2 [2] N2V19 §4.1 [3] P2N25 [4] Restormer22 Table 4 [5] reading

What we would deploy, and on what evidence

The brief asks what the current industry and academic standard is. The honest answer is that it depends on one constraint, and the fork in this deck's timeline predicts which.

■ Real photographs, GPU available, paired data exists	NAFNet . 40.30 dB on SIDD, best accuracy-per-millisecond measured, and by a wide margin the simplest code in Part VI — no attention, no activation functions. [1][7]
■ CPU-bound or memory-bound deployment	SCUNet . 9.663 M parameters, smallest of the architecture branch, and its synthetic-degradation training makes the blind weights unusually robust on photographs that were resized or JPEG-compressed before you ever saw them. [7][2]
■ Maximum quality on repetitive structure	Restormer , or MambaIR if CUDA is guaranteed. Restormer leads Urban100 at every noise level and is the only model past 40 dB on both SIDD and DND; MambaIR takes DND by 0.01 dB. Accept 10.4 s per 256 ² image on CPU. [3][4][7]
■ No clean data exists at all	P2N , or ZS-N2N for a single image with no training corpus and a minute of CPU. Accept per-image test-time optimisation as the price, and accept a quality gap to supervised training that is real and measured. [5][6]
■ A fast, dependency-free baseline	BM3D . Seventeen years old, needs no data and no GPU, still appears as a live baseline in 2023 papers, and still beats an AWGN-trained DnCNN on real sensor noise by 1.99 dB. [3]

01 **Notice what this list does not contain: a winner.** Five constraints, five different answers, all current. That is not indecision — it is what an unresolved research question looks like when you have to ship something on Monday.

Four current methods, two historical baselines — each strong on a different axis

A state-of-the-art claim is only checkable when the benchmark and metric are named, alongside what distinguishes each method from the others leading elsewhere. This table supplies both, and declines to estimate what no source reports.

METHOD	LEADS ON	FIGURE	PUBLISHED	WHAT DISTINGUISHES IT	COST [MEASURED] ^[5]
NAFNet (2022) ^[1]	SIDD — real sensor noise	40.30 dB	ECCV 2022	No attention and no activation functions at all; by far the simplest code	116.0 M · 1.90 s CPU
Restormer (2022) ^[2]	Urban100 — long-range structure	28.33 dB σ=50	CVPR 2022	First past 40 dB on both SIDD and DND; channel attention at linear cost	26.1 M · 10.44 s CPU
MambaIR (2024) ^[3]	DND — held-out real noise	40.04 dB	ECCV 2024	Global receptive field at linear cost via a selective state-space scan	will not import without CUDA
SCUNet (2023)	Robustness to synthesised real-world degradation	no single-table lead	Mach. Intell. Res. 2023	Competes on the training distribution rather than the architecture	9.7 M · 1.69 s CPU
DnCNN (2017) ^[4]	Historical — BSD68 Gaussian	29.23 dB	IEEE TIP 2017	The reference line the whole modern field was built against	0.67 M · 1.02 s CPU
BM3D (2007) ^[4]	Zero-shot — no training data	28.57 dB	IEEE TIP 2007	Requires no training data and no GPU; still the zero-shot baseline in N2N18, N2V19 and ZS-N2N23	no training · 2.85 s

No source states a SOTA window, so this deck does not assert one. The published date is verifiable; which table a method leads is verifiable from the cited comparison. For how long it held that position is not reported by any paper here, and we do not estimate it. Read the *Leads on* column: these methods are not ranked on one axis — they are strong on **different axes**, and those benchmarks measure different capabilities.

^[1] NAFNet22 abstract ^[2] Restormer22 Tables 4, 6 ^[3] MambaIR24 Table 6 ^[4] DnCNN17 Table II ^[5] measured, Stage 2

Four reasons the field has not converged on one method

Newer does not automatically mean universally better. 'Best' is only defined relative to a benchmark, a cost metric and a supervision regime — and the current leaders differ on all three.

REASON 1

The benchmarks measure different capabilities

BSD68 measures average-case Gaussian removal and **has saturated** — 0.36 dB in five years. Urban100 measures long-range self-similarity: the same span bought 1.98 dB. SIDD measures real noise: sixteen.^[1]

REASON 3

Deployment constraints are not quality constraints

MambaIR^[3] leads DND by 0.01 dB and **cannot be imported without compiled CUDA**. SCUNet leads no AWGN table and is a third of Restormer's size.^[5]

REASON 2

The cost axes disagree with each other

Our measurements give **three incompatible rankings** of the same models — by parameters, by CPU wall-clock, by real-noise quality. NAFNet^[2] has 4.4× Restormer's^[1] parameters and 5.5× its speed.^[5]

REASON 4

Supervision availability partitions the field

Where clean pairs exist, supervision wins by 1.35 dB.^[4] Where they do not — cryo-TEM, microscopy — self-supervised methods are not *worse*, they are the **only** option.

^[1] Restormer22 Table 4 ^[2] NAFNet22 ^[3] MambaIR24 ^[4] N2V19 §4.1 ^[5] measured, Stage 2

Every step, as an implementation change

The same evolution read through the code rather than the papers. Each row names the smallest change that carries the innovation — the connection from contribution, through algorithmic change, to implementation change, to measured improvement.

STEP	BEFORE	AFTER	THE CHANGE IN CODE ^[4]	MEASURED EFFECT
BM3D → WNNM (2014)	Shrink a 3-D transform of the patch group	Recover a low-rank matrix from it	Transform call becomes an iterated SVD with per-singular-value weights	+0.26 dB · 271× slower ^[1]
WNNM → TNRD (2016)	<code>while</code> loop to a tolerance, one fixed rule	Fixed K stages, each with its own learned filters	Loop becomes a fixed stack trained by back-propagation	+0.09 dB · 773 s → 0.032 s ^[1]
TNRD → DnCNN (2017)	Stages derived from a diffusion PDE	Plain Conv-BN-ReLU ×17, no pooling	<code>return self.model(x)</code> becomes return x - self.model(x)	+0.31 dB · one blind model ^[1]
DnCNN → N2N (2018)	<code>target = clean</code>	<code>target = second_noisy_shot</code>	One assignment. Network, optimiser, schedule unchanged	−0.02 dB · clean data no longer needed ^[2]
N2N → N2V (2019)	Two noisy shots of one scene	One noisy image	Replace the centre pixel with a random neighbour; score the loss only there	−1.15 dB · applicable where no pair exists ^[3]

Read the fourth column down the page. An iterated SVD, then a training loop, then a single subtraction, then one assignment, then a masking trick. The size of the code change is uncorrelated with the size of the conceptual change — which is why reading the repositories was worth doing.

^[1] DnCNN17 Tables II, IV ^[2] N2N18 Table 1 ^[3] N2V19 Fig. 4 ^[4] code measured at the commits on slide 65

The architecture branch, as an implementation change

The same treatment for the three operators that answer bottleneck 3, plus the one method that answers it from the data side instead.

STEP	BEFORE	AFTER	THE CHANGE IN CODE ^[5]	MEASURED EFFECT
Spatial → Restormer ^[1] (2022)	Attention over HW×HW pixels — 4.3×10 ⁹ entries at 256 ²	Attention over C×C channels — 48×48 at level 1	One einops pattern: 'b (head c) h w -> b head c (h w)' puts pixels on the last axis	Linear in pixels · +1.98 dB on Urban100 vs DnCNN
Attention → NAFNet ^[2] (2022)	GELU, then a gate; CA with ReLU and Sigmoid	The gate <i>is</i> the nonlinearity	<code>x1, x2 = x.chunk(2, dim=1); return x1 * x2</code>	+0.08 dB over GELU · 40.30 dB SIDD
Attention → MambaIR ^[3] (2024)	Quadratic attention, or windows that surrender globality	Selective state-space recurrence	Flatten to a sequence, four-directional scan , plus a local conv and channel attention	Linear cost · best DND at 40.04 dB
Architecture → SCUNet ^[4] (2023)	Train on AWGN, test on AWGN	Train on a synthesised degradation pipeline	The dataloader becomes the contribution: Poisson, speckle, JPEG, ISP-simulated sensor noise, resizing	9.7 M params · robust to unseen degradation

And the honest footnote from NAFNet's own ablation: the largest single gain in its table is not an architectural component at all — it is **LayerNorm**, **+0.44 dB**, because it stabilises training at a ten-times larger learning rate. A substantial share of reported architectural progress in restoration is optimisation progress wearing a costume.

^[1] Restormer22 §3.1, Tables 4, 7 ^[2] NAFNet22 §4, §5 ^[3] MambaIR24 §3.2 ^[4] SCUNet23 §3.2 ^[5] code measured

Four research directions the limitations actually justify

Each is tied to a specific documented gap — from the papers' own words or from our measurements — and each states what would change and how it would be measured. None claims a paper achieved something it did not.

DIRECTION	GROUNDING IN	WHAT WOULD CHANGE	HOW YOU WOULD MEASURE IT
Cross the fork — self-supervised supervision on a modern backbone	Our own finding: no published work trains a Transformer or SSM backbone under RDC/DCS supervision; P2N still ships the 2018 U-Net verbatim[1]	Replace UNet_n2n_un with a Restormer or NAFNet block under P2N's RDC/DCS training signal	N2V's own protocol on BSD68 $\sigma=25$ and on the cryo-TEM / CTC sets — does the 1.35 dB self-supervision gap shrink?
A cross-ISP generalisation benchmark	SCUNet's abstract[2]: a general-purpose blind real denoiser <i>remains unsolved</i> , and no benchmark evaluates an unseen camera	Train on SIDD, evaluate on DND <i>and</i> on a held-out camera; report the drop as the headline number	Δ PSNR between seen and unseen ISP — a quantity nobody currently reports
A perceptual objective that does not surrender fidelity	Noise2Noise[3] names L2's <i>spatial blurriness</i> ; perceptual losses are standard in super-resolution and essentially absent from denoising	Add an LPIPS or no-reference term to NAFNet's PSNR loss and report both axes rather than one	PSNR <i>and</i> LPIPS jointly on SIDD — accept a small dB loss for a measured perceptual gain
An efficiency protocol the field can agree on	Three incompatible rankings from our own measurements[5]; MambaIR's changelog[4] conceding its published <i>thop</i> figures were wrong	Report parameters, MACs <i>and</i> wall-clock on a named device, with the counting method stated alongside	Reproduce the table across CPU and GPU; the change in ranking is itself the result

The one we would pursue is the first. It falls directly out of the structural claim this deck has carried since Part IV — the two branches have never merged — it is cheap to test because both halves already exist as working code, and it has a clear negative result if the gap does not close.

[1] reading, grounded in P2N25 [2] SCUNet23 abstract [3] N2N18 §2 [4] MambaIR24 changelog [5] measured, Stage 2

The condensed executive summary, and what it leaves out

The brief is explicit that a four-hour presentation is impossible: you present a highly condensed executive summary and are questioned on any part of the full deck. This slide states how that summary is cut, so the choice is deliberate rather than improvised.

What the 26-minute summary covers

- **The claim, before the evidence** — four eras, three bottlenecks, one fork in 2018.
- **One Before → After per era**, each with the implementation change named: the iterated SVD, `return x - n`, the target swap, the einops rearrange, `x1 * x2`.
- **The three bottleneck slides**, because they are what make the timeline evidenced rather than chronological.
- **The SOTA comparison and why several coexist**, which is the question most likely to be asked.
- **Two presenters**, handing over at the bottleneck slide — the seam where one era's debts become the next era's branches.

What it deliberately leaves to the deck

- The full classical derivations — MAP, total variation, the WNNM weighting theorem.
- The engineering detail of Part VI: padding constants, framework migration, checkpoint loading, the full measured benchmark.
- Per-paper training recipes, dataset tables and the complete bibliography.
- **These are exactly the parts you will be questioned on**, which is why the rule stands: do not put anything on a slide you cannot personally explain.
- The condensed deck and its run sheet are generated from this deck's own content, so the two cannot drift apart.

Per the brief: the deck is the standalone educational artifact and carries the depth; the presentation is a condensed executive summary of it. Both are in this submission, and the summary's backup index maps every anticipated question to a slide number in this deck.

The thesis, restated

Image denoising is the cleanest available case study in how computer vision replaced hand-designed priors with learned ones.

Its history divides into four eras separated by three identifiable bottlenecks — manual prior design, the clean-data requirement, and the receptive-field limits of convolution. Each bottleneck was named by the paper that broke it, so the arc is evidenced from the primary sources rather than imposed in hindsight.

In 2018 the response forked. One branch changed what the network is shown; the other changed what the network is. They have never merged.

And because the field currently holds three competing answers to the third bottleneck and none to the first open problem, the story ends in a genuine open question rather than a winner.

— The claim this deck has spent 100 slides evidencing.

Category A — twelve peer-reviewed papers

Citation counts are Google Scholar figures checked on 9 August 2026, except P2N, which was not verified. Counts move continuously; the date is quoted alongside every figure.

#	PAPER (SHORT TITLE)	AUTHORS	VENUE & YEAR ^[2]	CITED BY ^[1]
A1	Sparse 3-D Transform-Domain Collaborative Filtering	Dabov, Foi, Katkovnik, Egiazarian	IEEE TIP 16(8), 2007	11 976
A2	Weighted Nuclear Norm Minimization	Gu, Zhang, Zuo, Feng	CVPR 2014	2 960
A3	Trainable Nonlinear Reaction Diffusion	Chen, Pock	IEEE TPAMI 39(6), 2017	1 626
A4	Beyond a Gaussian Denoiser	Zhang, Zuo, Chen, Meng, Zhang	IEEE TIP 26(7), 2017	11 674
A5	Noise2Noise	Lehtinen, Munkberg, Hasselgren et al.	ICML 2018	3 064
A6	Noise2Void	Krull, Buchholz, Jug	CVPR 2019	2 075
A7	Restormer	Zamir, Arora, Khan et al.	CVPR 2022	6 181
A8	Simple Baselines for Image Restoration	Chen, Chu, Zhang, Sun	ECCV 2022	2 318
A9	Zero-Shot Noise2Noise	Mansour, Heckel	CVPR 2023	266
A10	Practical Blind Denoising via Swin-Conv-UNet	Zhang, Li, Liang, Cao et al.	Mach. Intell. Research 20(6), 2023	391
A11	MambaIR	Guo, Li, Dai, Ouyang, Ren, Xia	ECCV 2024	1 208
A12	Positive2Negative	Li, Wang, Xu, Zhu, Lu, Huang	CVPR 2025, pp. 17924–17934	not verified

Two observations worth making aloud rather than leaving in the table. BM3D and DnCNN sit near twelve thousand citations each, which is precisely why both remain universal baselines nineteen and nine years after publication — the counts corroborate the narrative rather than decorating it. Conversely ZS-N2N at 266 and SCUNet at 391 are low because they are recent, not because they are weak; recency and citation count are confounded, and saying so pre-empts the obvious challenge.

^[1] Google Scholar, checked 9 August 2026 ^[2] venues verified per slide 100

Category B — three auxiliary sources, and why each is here

Three of fifteen sources, or 20.0%, against the brief's 30% ceiling. Category A is 80.0%, against a 70% floor. Each Category B item is included because it does something no paper in the roster does.

-
- 01 **B1 • Stanford CS231n**, *Convolutional Neural Networks for Visual Recognition*. A recognised academic lecture series, explicitly permitted by the brief. It supplies the backpropagation, convolution-arithmetic and optimisation background that every Category A paper from A4 onward assumes without stating. **Role in the narrative:** it is the reason slides 32–34 can discuss receptive fields and batch normalisation as shared vocabulary rather than deriving them.
 - 02 **B2 • Google Research**, *Night Sight / HDR+* technical blog. A highly technical developer publication from an industry leader, explicitly permitted by the brief. It separates photon shot noise from read noise, explains that SNR rises with the square root of exposure time, and describes how a burst of frames is captured, aligned in software and merged. **Role in the narrative:** it is the only source in the roster describing denoising as a *shipped consumer product*, and it supplies the physical grounding for the real-noise argument on slide 83 and 88.
 - 03 **B3 • Gonzalez & Woods**, *Digital Image Processing* — the Image Restoration chapter. A textbook chapter, explicitly permitted. It supplies the formal treatment of noise models (Gaussian, Rayleigh, Poisson, salt-and-pepper) and of mean, order-statistic and adaptive filters that Part II's first three slides rest on. **Role in the narrative:** it is the bottom rung of the timeline, below BM3D, and it is the reference against which SCUNet's synthetic degradation pipeline reads critically — SCUNet's noise list is essentially the textbook's noise list, industrialised.
 - 04 **Verification note.** Chapter number and page range differ between the 3rd and 4th editions of Gonzalez & Woods. Cite the exact edition physically in use.
 - 05 **The arithmetic.** 15 sources total. 12 Category A = 80.0% (floor 70%). 3 Category B = 20.0% (ceiling 30%). Compliant with margin on both sides.

Declared limitations — stated, not discovered

A gap that is stated is a smaller problem than a gap that is found. These are every known weakness in this deck.

-
- 01 **No quality metric in this deck was computed by us.** Every PSNR and SSIM is reproduced from a published table. Our own measurements cover parameter counts and CPU forward-pass latency only, and used **random tensors**, so they carry no image-quality information whatsoever.
 - 02 **CPU latencies are not comparable to published GPU timings.** Only the ratios between our own measurements are meaningful, and they are meaningful only for the CPU backend we used.
 - 03 **The DnCNN runtime table mixes devices.** Classical methods are timed on CPU, learned methods on GPU, and the paper states that GPU times for CSF and TNRD were copied from their original papers. We reproduce this caveat wherever we reproduce the numbers.
 - 04 **One published value is excluded.** MambaIR's CBS68 $\sigma=25$ figure extracted as 32.24, which is inconsistent with its own row (34.48 at $\sigma=15$, 28.66 at $\sigma=50$) and with every comparable method. We judged this a probable extraction error and excluded the cell rather than reproduce it. MambaIR's colour results are quoted from Urban100 instead, which is internally consistent.
 - 05 **Venue verification.** Venues for BM3D, TNRD, Noise2Noise and SCUNet are printed in the source PDFs themselves. The remaining eight are preprint versions carrying no venue, and were confirmed against publisher, CVF Open Access or institutional proceedings records rather than copied from another paper's reference list.
 - 06 **TNRD's year is genuinely ambiguous.** The PDF header reads 2016; the citing literature gives TPAMI 39(6), 2017. Both reflect reality — acceptance versus issue. We cite 2017.
 - 07 **Reference code availability is uneven.** ZS-N2N has no repository, only a Colab notebook. WNNM has only a third-party mirror with no README or licence. TNRD's available implementation is a same-laboratory re-implementation, not the authors' original release. The P2N paper's printed repository URL returns HTTP 404; the live code is at [P2N-plus](#).
 - 08 **SCUNet's venue is not on the brief's named list.** *Machine Intelligence Research* is a peer-reviewed Springer journal but is not among the venues the brief names. The brief's list is illustrative rather than exhaustive; we flag this rather than conceal it.

Where to find the answer to the question you are about to be asked

The deck is indexed by likely question rather than by topic, because that is how it will be interrogated.

IF THEY ASK...	GO TO	THE ONE-LINE ANSWER
Why is 0.6 dB a big deal?	10, 37	Practical ceiling over BM3D is 0.3 dB; theoretical bound 0.7 dB.
Why residual learning?	33	y is closer to x than to v, so F(y) would approximate an identity — the hard point in parameter space.
Why do RL and BN need each other?	34	RL makes layer inputs Gaussian-like and content-independent, which is what BN's batch statistics require.
Why depth 17?	35	RF (2d+1) ² set to 35×35 to match EPLL, the smallest effective patch size in the field.
Where does DnCNN lose, and why?	38, 51	Barbara, −0.71 dB. Repetitive structure meets a non-local prior a 35×35 field cannot reach.
How can noisy targets work?	43	L2 learns the conditional mean; the mean of a zero-mean corrupted target is the clean target.
Why does the blind spot not learn identity?	45	Neighbours carry no information about a pixel's own noise, given conditional pixel-wise independence.
Where is Restormer's attention linear?	69	<code>restormer_arch.py</code> , <code>Attention.forward</code> — the <code>rearrange</code> puts pixels last, so <code>q @ k.transpose(-2,-1)</code> is C×C.
Why is NAFNet faster with more parameters?	77	1×1 and depth-wise convolutions hit optimised GEMM paths; <code>rearrange</code> forces layout changes.
What did the code tell you that the papers did not?	75, 77	Padding differs 8×; SPI cuts parameters and triples latency; MambaIR needs CUDA to import.
Which model would you deploy?	89	Name the constraint first. Five constraints, five different answers, all current.
What is still unsolved?	87, 88	Blind real denoising, perception–distortion, and an agreed cost metric.

The rule for the defence. Do not put anything on a slide you cannot personally explain. Every slide in this deck was built to be answered from, and the notes field of every slide carries the answer it expects.